CUDA : étude de performance

Utilisation d'un LLM

J'ai utilisé une LLM (Claude Sonnet 5.5) pendant ce travail. Voici à quoi elle m'a servi :

- Installer et vérifier l'environnement CUDA (CUDA Toolkit, compilateur MSVC, script `build.bat`).
- Corriger les erreurs trouvées en testant (par exemple le type `long` de 32 bits sous Windows).
- Reprendre la structure du notebook et le stockage des mesures du TP OpenMP (classeur `mesures_cuda.xlsx`, ajout des résultats après chaque lancement, reprise sans doublons).
- Étoffer les discussions (valeurs chiffrées, calculs de facteurs et parfois correction des hypothèses d'explication).
- Aide dans l'écriture des descriptions des programmes avant les sections code.
- La mise en place du chrono.

Environnement expérimental

GPU	NVIDIA GeForce GTX 1650 (architecture Turing, capacité de calcul 7.5, 14 multiprocesseurs), 4 Go de mémoire
Pilote	610.60
Compilateur	nvcc 13.4 (CUDA Toolkit 13.4) avec MSVC (Visual Studio Build Tools 2022)
Options	`-O3 -arch=sm_75` (script `build.bat`)
Système	Windows 11 Famille

Méthodologie : chaque configuration est lancée au moins 10 fois par un script Python (bibliothèque `subprocess`), sauf aux très grandes tailles de la partie 1 (voir 1.5). Chaque exécution écrit une ligne avec ses paramètres, qui est ajoutée à une feuille du classeur `mesures_cuda.xlsx` (une feuille par partie). Les tableaux donnent la moyenne ± l'écart-type, et les figures tracent la moyenne avec des barres d'erreur (écart-type) ainsi que les mesures individuelles. Les paramètres (nombre de pas, threads par bloc, pas par thread) sont des arguments des programmes, sans recompilation. Le temps mesuré par les programmes comprend le lancement des kernels, les réductions et la copie du résultat vers le CPU, mais pas l'initialisation du contexte CUDA ni les allocations.

Pour reproduire : exécuter le notebook de haut en bas sur une machine avec une carte NVIDIA, le CUDA Toolkit et Python (pandas, matplotlib, openpyxl).

Organisation des fichiers

- `src/` : les sources (`.cpp` pour la référence séquentielle, `.cu` pour CUDA)
- `bin/` : les exécutables compilés (créé par le notebook)
- `donnees/` : le classeur `mesures_cuda.xlsx` (une feuille par partie)
- `build.bat` : compile un fichier `.cu` avec nvcc (`.\build.bat src\fichier.cu`)
- `rapport_CUDA.pdf` : la version PDF de ce rapport

Stockage des mesures

Les mesures sont rangées dans le classeur `donnees/mesures_cuda.xlsx`, avec une feuille par partie. Chaque programme écrit ses résultats dans un petit fichier CSV temporaire, que la cellule suivante ajoute au classeur puis supprime. Les cellules de lancement ne relancent que les mesures qui manquent pour atteindre le nombre de répétitions voulu.

1. Computing PI

Référence séquentielle

Code de départ fourni (non modifié), utilisé comme référence pour les comparaisons de performance dans les questions suivantes. Il calcule pi par intégration de 4/(1+x^2) sur [0,1] : chaque itération est indépendante, seule l'accumulation finale dans `sum` crée une dépendance.

Question 1.1 - Port CUDA avec un thread par bloc

Le calcul est confié à un kernel lancé avec `N_blocs` blocs, chacun associé à un seul thread. Le tableau est découpé en tranches de `S` pas (`S` = pas par thread, argument `-S`) et le bloc numéro `b` calcule les pas `b*S + 1` à `(b+1)*S`. Chaque thread écrit sa somme partielle dans un tableau en mémoire du GPU, copié ensuite vers le CPU où la somme finale est faite. Le nombre de blocs est `(N+S-1)//S`.

Remarques :

- Le type `long` ne fait que 32 bits sous Windows, alors que 10^10 et 10^12 dépassent 2^31 : donc les compteurs ont été modifiés en `long long`.
- Le nombre de blocs est limité à 2^31 - 1 et la mémoire du GPU à 4 Go. Un programme qui ne peut pas s'exécuter s'arrête avec un code de sortie (2 : trop de blocs, 3 : mémoire insuffisante, 4 : erreur de lancement) et n'écrit aucune mesure.

Question 1.2 - Plusieurs threads par bloc (réduction à deux niveaux)

Chaque bloc contient maintenant `T` threads (argument `-T`). Le thread d'indice global `id = blockIdx.x * blockDim.x + threadIdx.x` calcule sa tranche de `S` pas et écrit sa somme dans un tableau en mémoire globale. Après un `__syncthreads()`, le thread 0 de chaque bloc additionne les `T` valeurs de son bloc (premier niveau) et écrit le total du bloc. Le CPU additionne les totaux des blocs (second niveau).

Question 1.3 - Mémoire partagée et atomiques

Chaque bloc a une variable `__shared__` (le total du bloc), initialisée à 0 par le thread 0. Chaque thread y ajoute sa somme avec `atomicAdd`, puis le thread 0 ajoute le total du bloc au résultat global avec un second `atomicAdd`. Il n'y a plus de tableau de sommes partielles ni de somme sur le CPU. Les deux `__syncthreads()` servent à garantir que le total est initialisé avant les ajouts, et est complet avant sa lecture.

Question 1.4 - Réduction multi-étapes complète

Il n'y a plus d'atomiques. Chaque thread écrit sa somme dans la mémoire partagée du bloc, puis le bloc fait une réduction en arbre : à chaque tour, la moitié des threads actifs ajoute la valeur de l'autre moitié, ce qui demande `log2(T)` tours (mais il faut supposer que `T` est une puissance de 2). Le premier kernel produit un total par bloc. Un second kernel (`reduce_kernel`) applique la même réduction au tableau de totaux, et on le relance tant qu'il reste plus d'une valeur. Deux tableaux servent tour à tour d'entrée et de sortie, car un bloc ne doit pas écrire dans une case qu'un autre bloc lit.

Question 1.5 - Évaluation de performance

L'énoncé demande d'évaluer le nombre de pas 10^6, 10^8, 10^10 et 10^12 avec 1, 32, 64, 128 et 256 threads par bloc et 1, 64, 256 et 1024 pas par thread. La grille complète de mesures a été faite sur la version finale (Q1.4). Les versions Q1.1, Q1.2 et Q1.3 sont mesurées sur la même grille mais seulement jusqu'à 10^8 pas, afin de les comparer dans la discussion. Pour la Q1.1, T vaut toujours 1 (un thread par bloc).

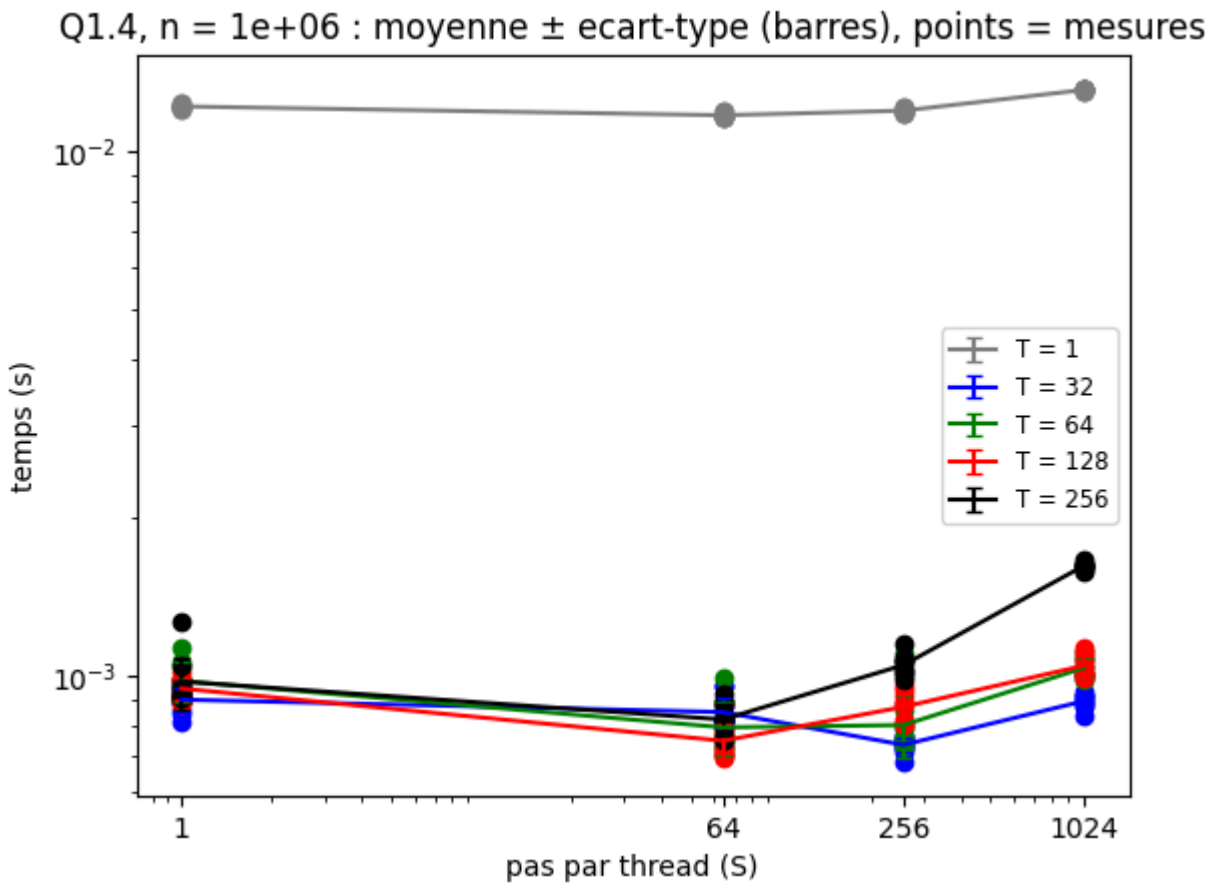
Limites :

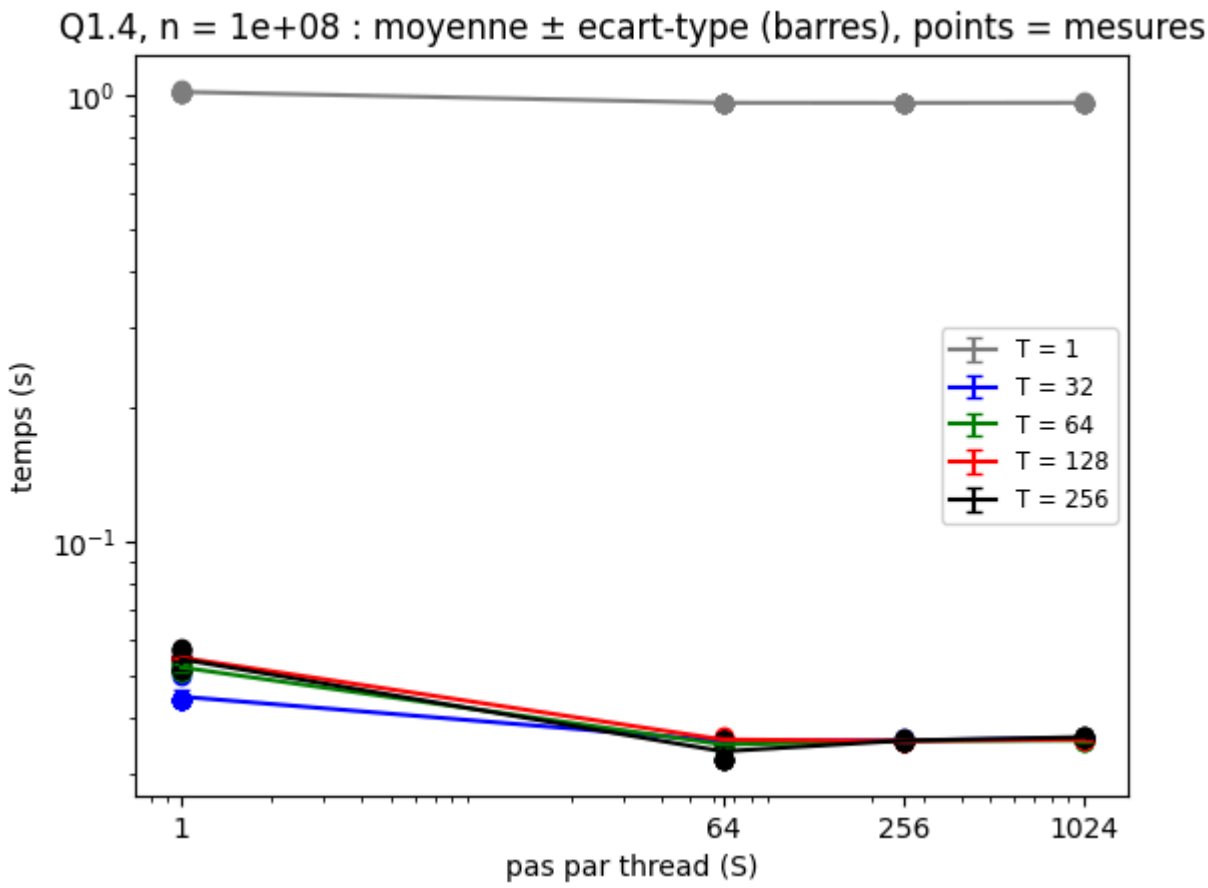
- **Blocs et threads** : le nombre de blocs ne peut pas dépasser 2^31 - 1. À 10^10 pas avec 1 pas par thread, il faut au moins 5 threads par bloc.
- **Mémoire** (4 Go) : la Q1.2 stocke un `double` par thread, ce qui dépasse la mémoire pour les très grands nombres de threads.
- **Durée** : lors des tests, j'ai pu constater qu'un thread par bloc est environ 30 fois plus lent. À partir de 10^10 pas, le choix a donc été fait de ne garder que T >= 32. À 10^12 pas (environ 5 minutes par exécution dans les meilleurs cas), on se limite à T = 32, 64, 128 et 256, deux valeurs de S (256 et 1024) et 2 répétitions.

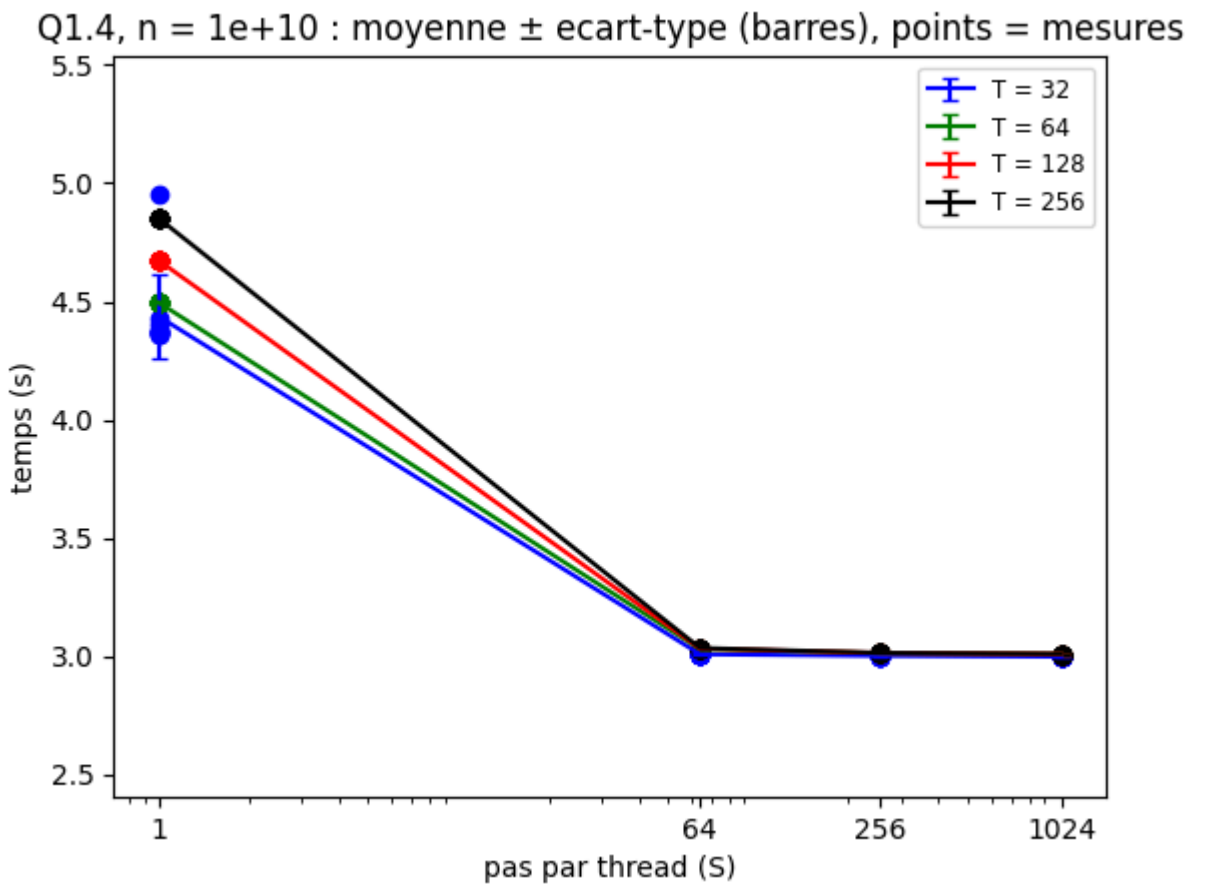
Les cas impossibles échouent immédiatement (codes de sortie 2, 3 ou 4) et ne produisent aucune ligne de mesure.

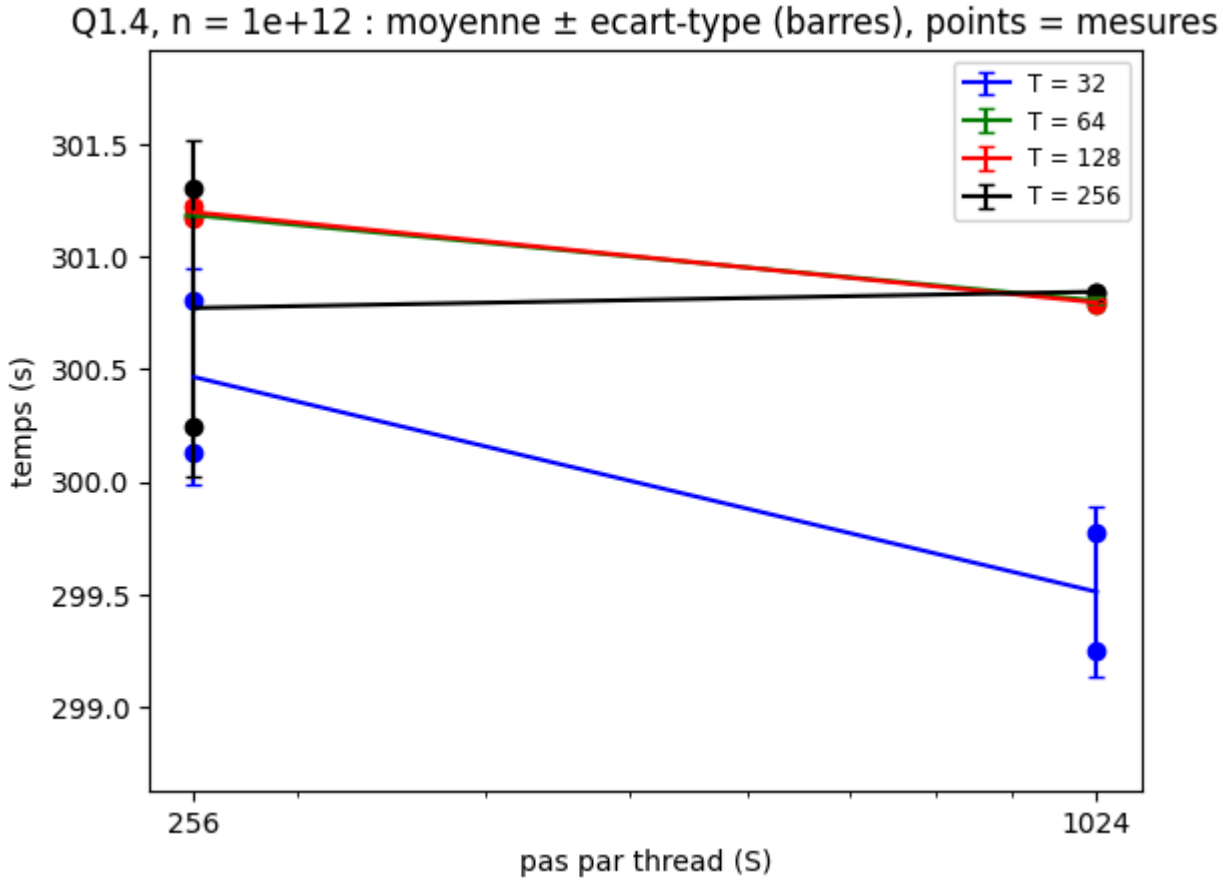
Question 1.6 - Figures de performance

Temps en fonction du nombre de pas par thread (Q1.4)

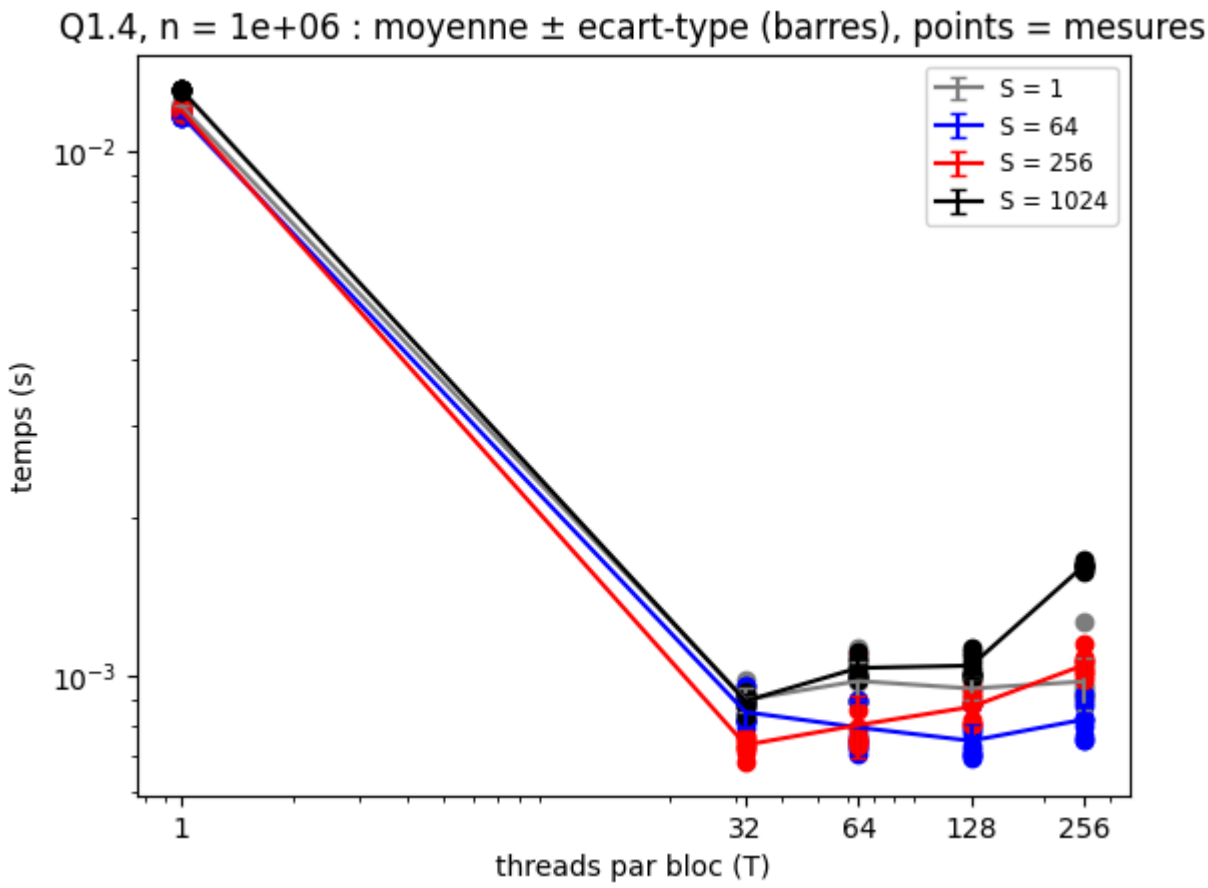

Q1.4, n = 1e+06 : moyenne ± ecart-type (barres), points = mesures
10^{-2}
10^{-3}
temps (s)
T = 1
T = 32
T = 64
T = 128
T = 256
1
64
256
1024
pas par thread (S)

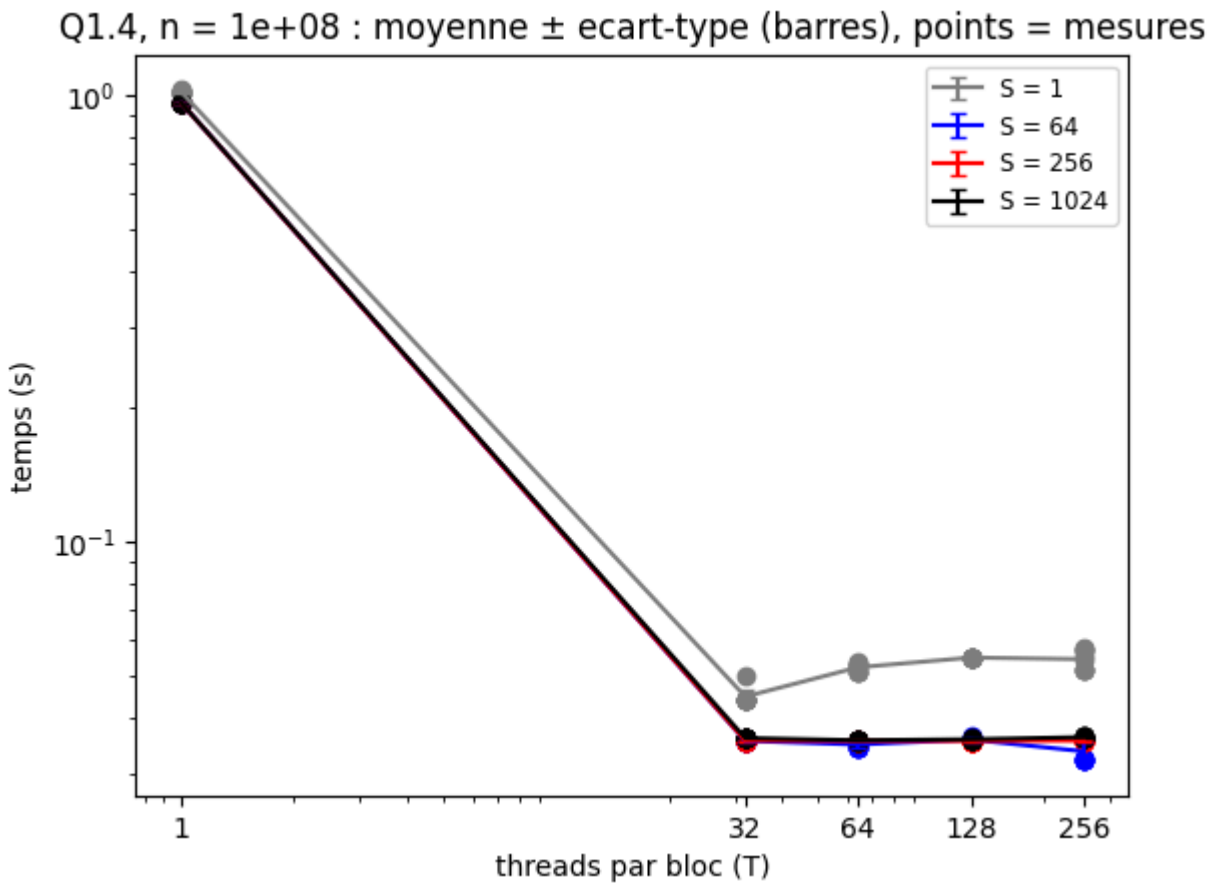

Q1.4, n = 1e+08 : moyenne ± ecart-type (barres), points = mesures
10^{0}
10^{-1}
temps (s)
T = 1
T = 32
T = 64
T = 128
T = 256
1
64
256
1024
pas par thread (S)

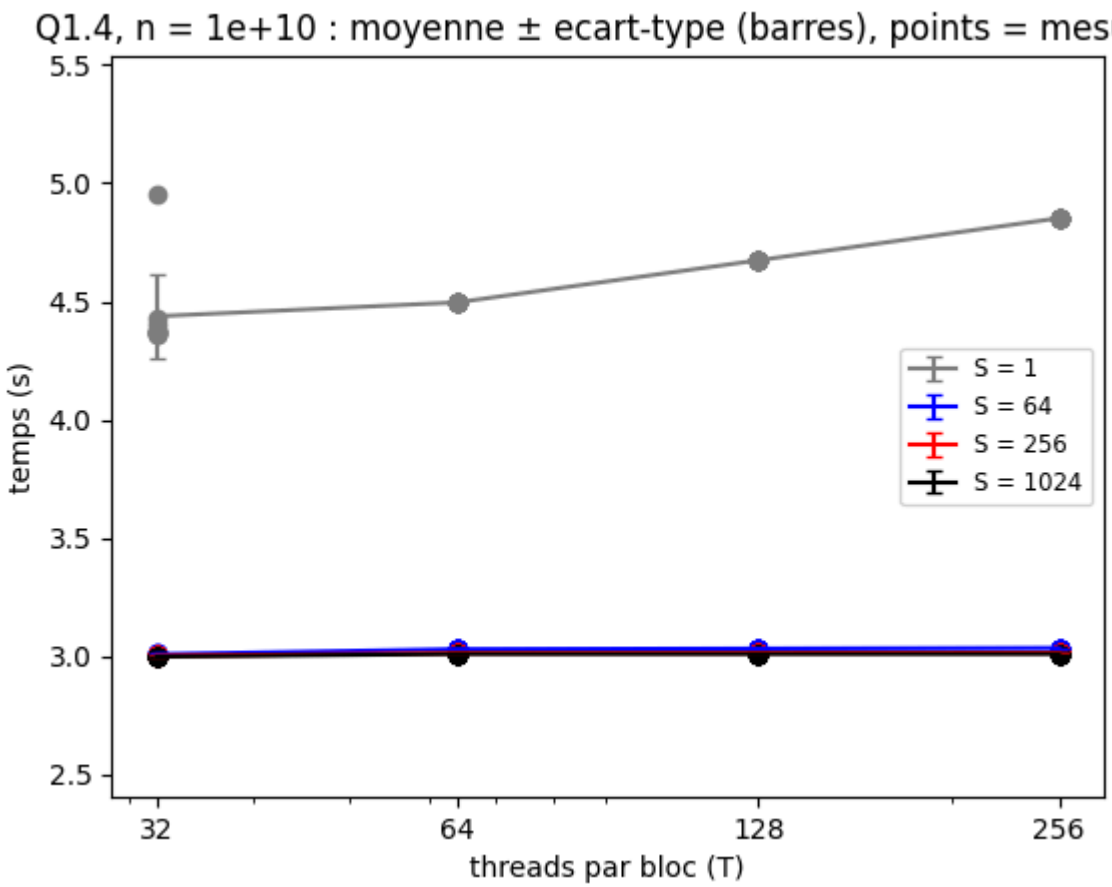


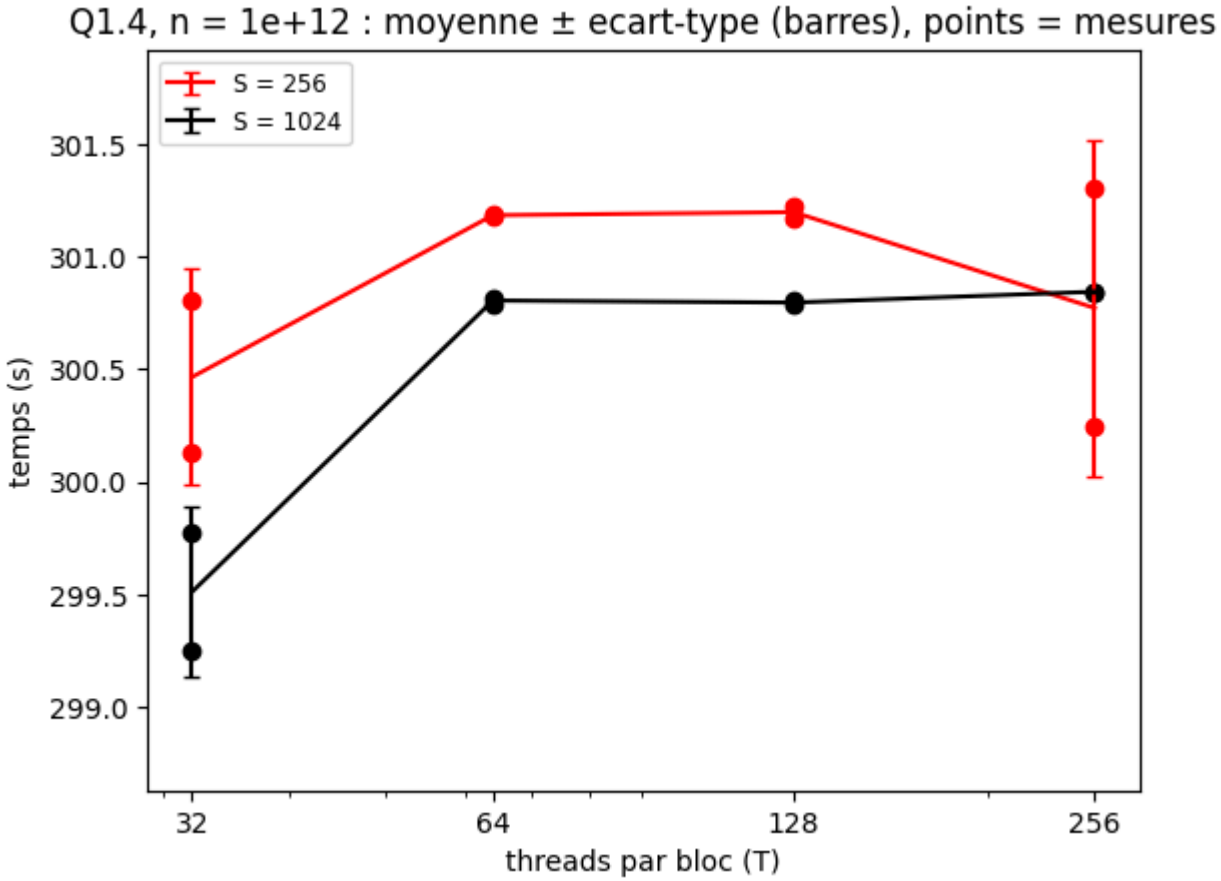


Temps en fonction du nombre de threads par bloc (Q1.4)

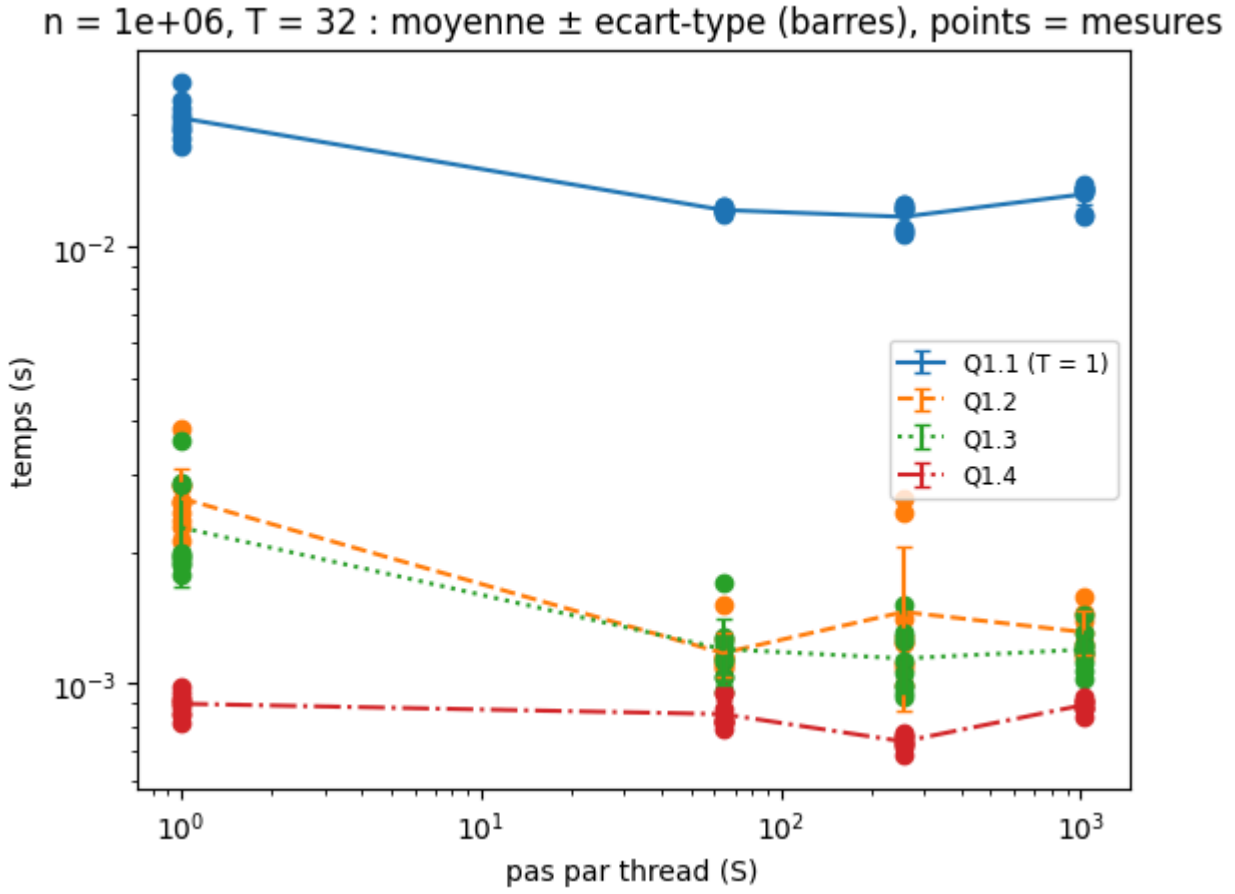

Q1.4, n = 1e+06 : moyenne ± ecart-type (barres), points = mesures
S = 1
S = 64
S = 256
S = 1024
10^{-2}
10^{-3}
temps (s)
1
32
64
128
256
threads par bloc (T)

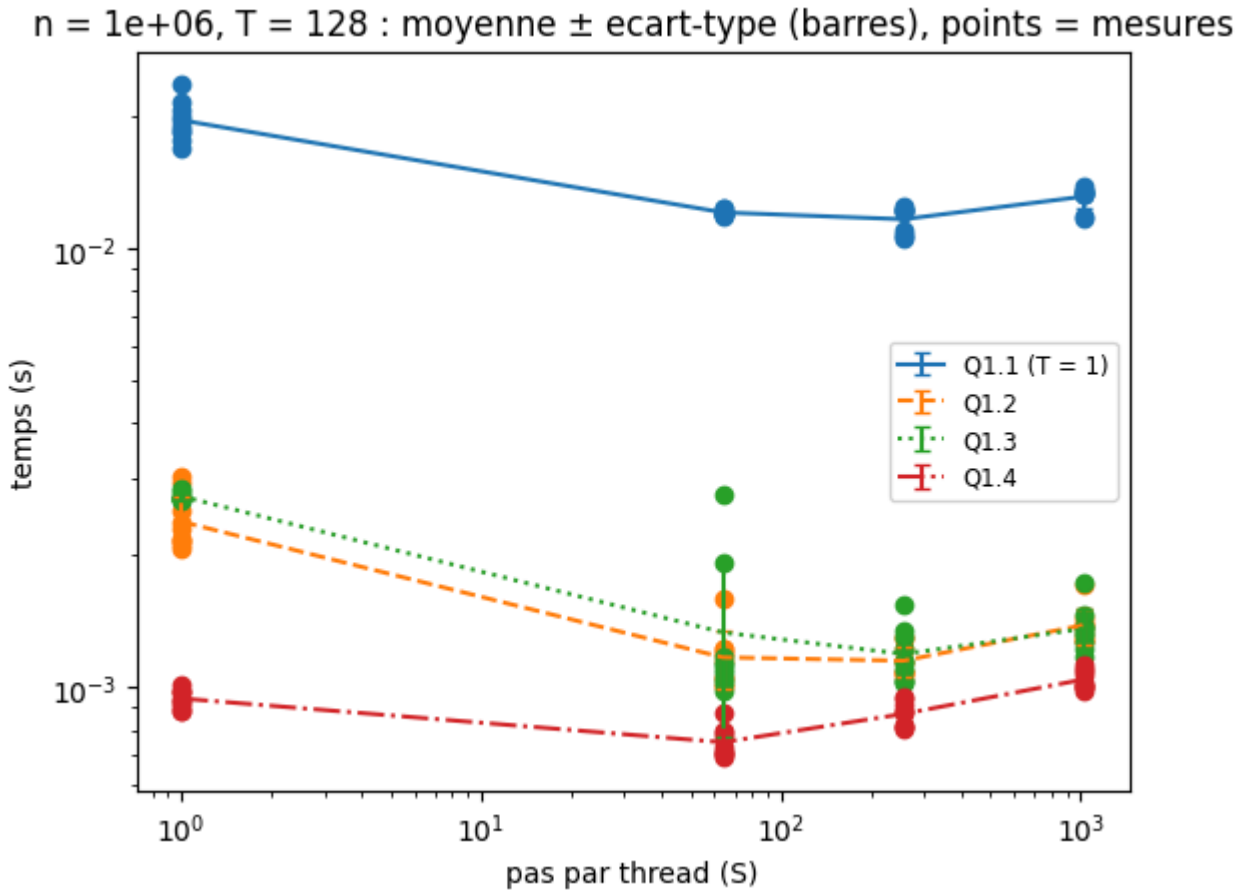

Q1.4, n = 1e+08 : moyenne ± ecart-type (barres), points = mesures
S = 1
S = 64
S = 256
S = 1024
10^{0}
10^{-1}
temps (s)
1
32
64
128
256
threads par bloc (T)

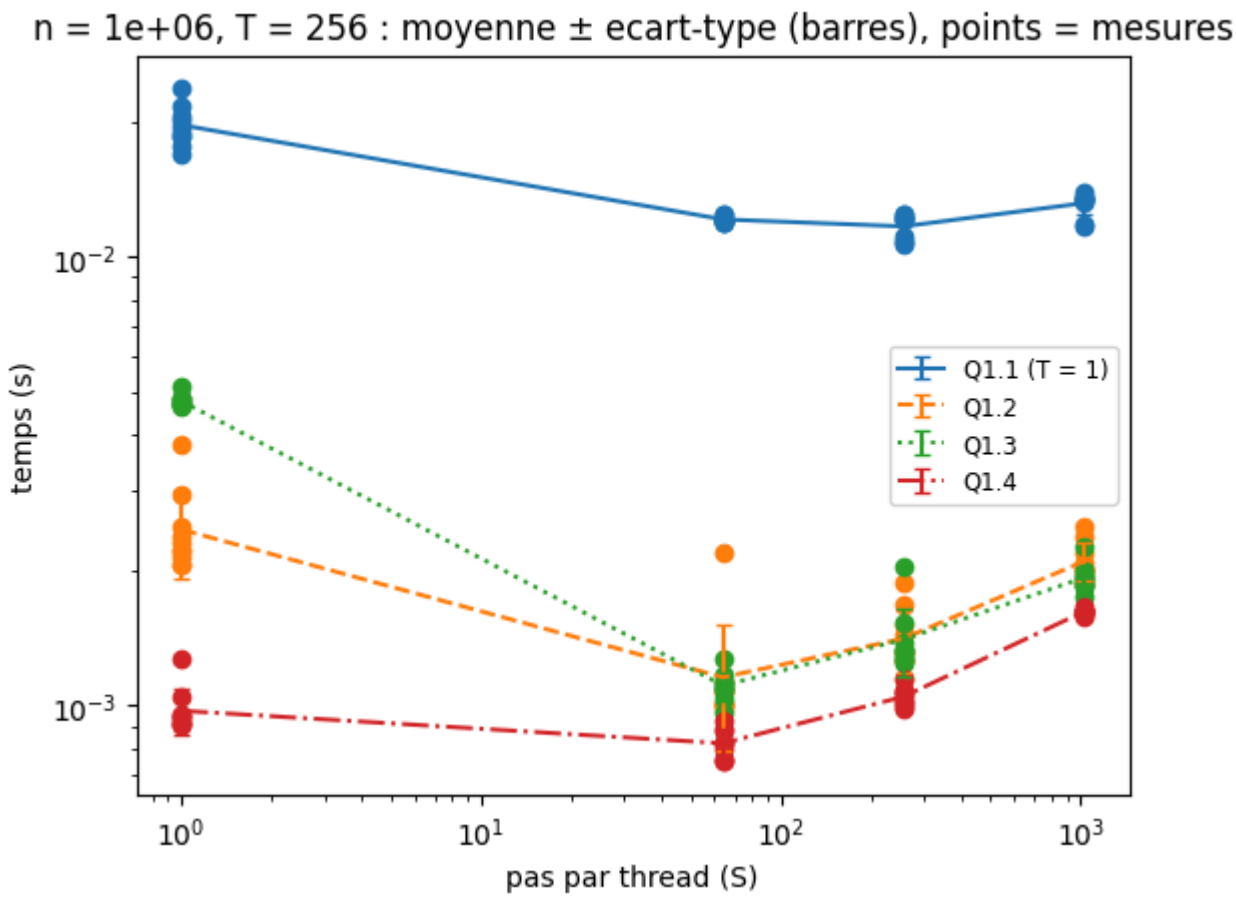


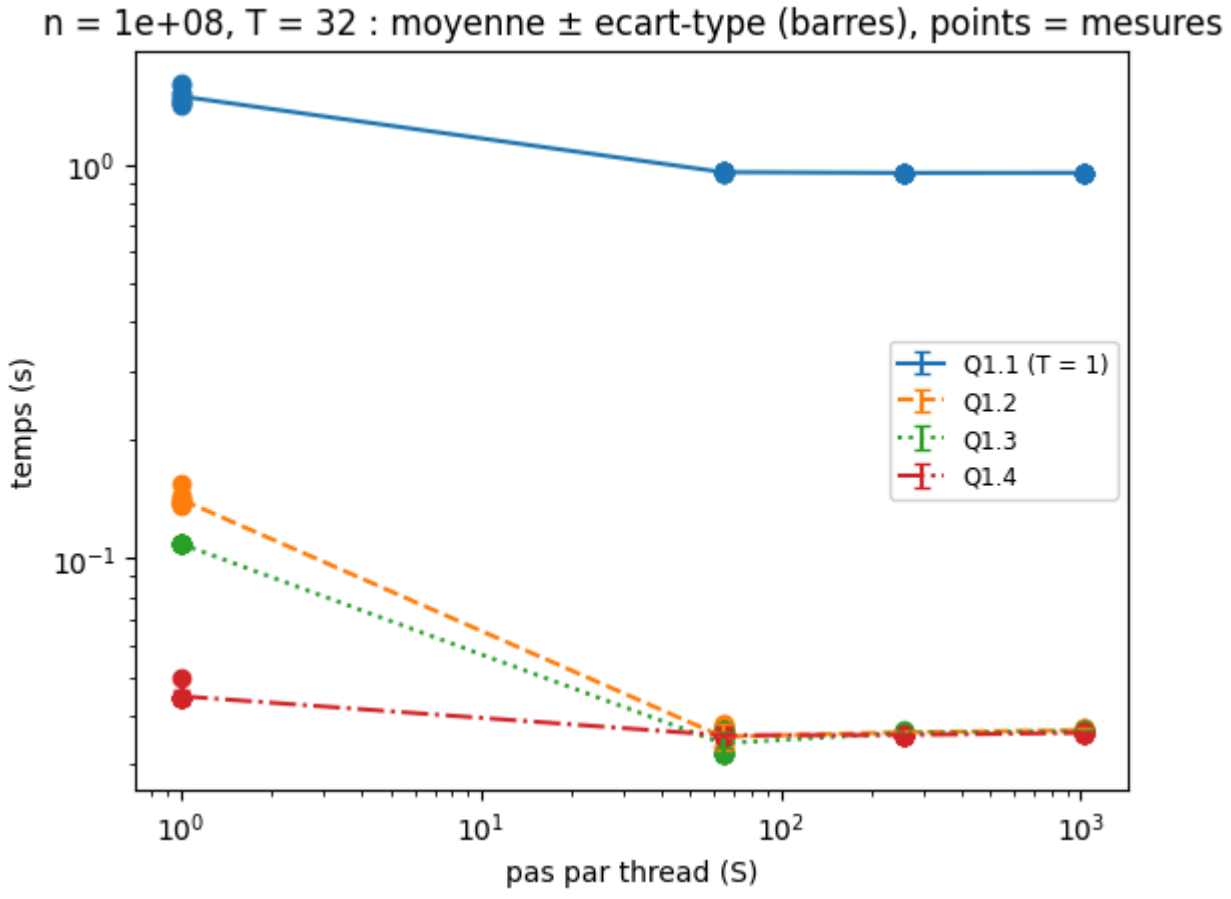


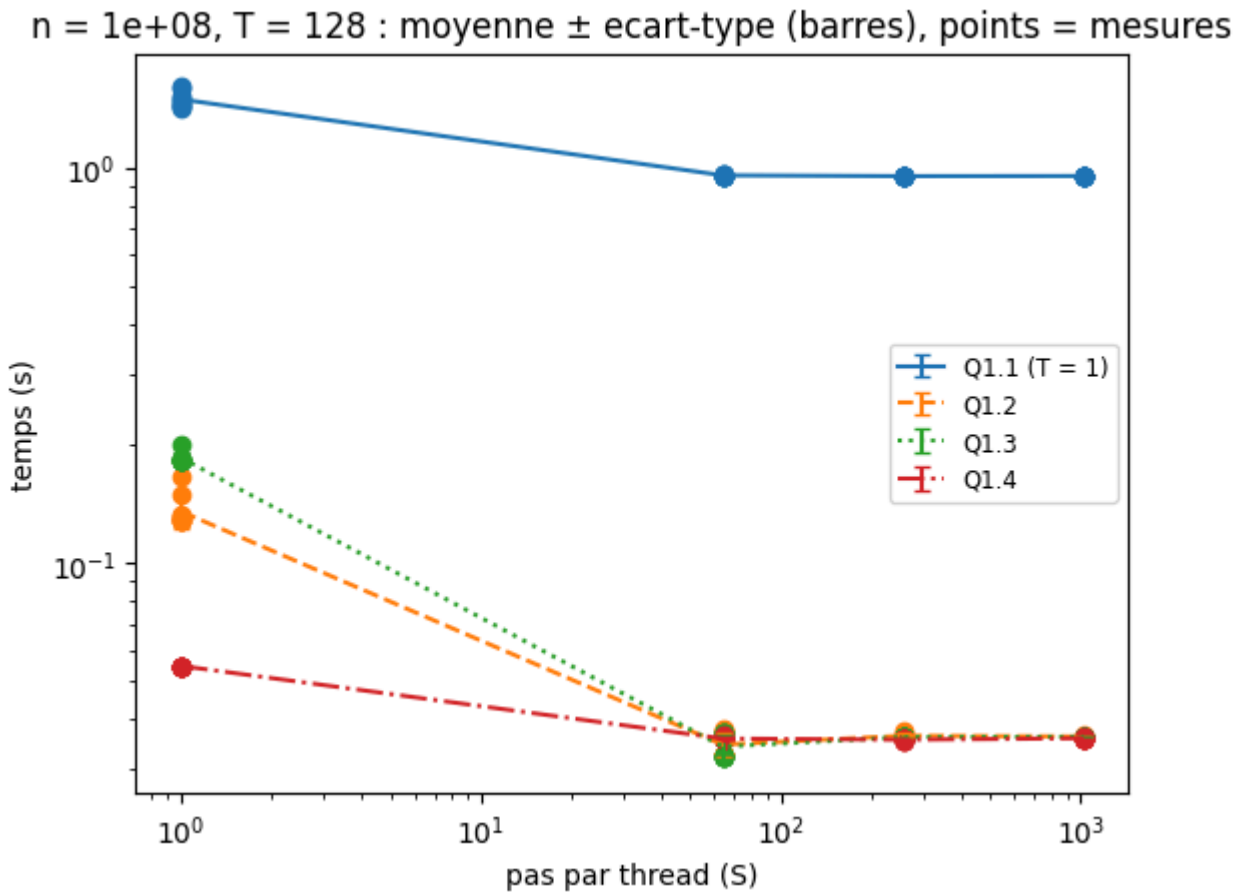
Comparaison des quatre versions


n = 1e+06, T = 32 : moyenne ± ecart-type (barres), points = mesures
temps (s)
10^{-2}
10^{-3}
10^{0}
10^{1}
10^{2}
10^{3}
pas par thread (S)
Q1.1 (T = 1)
Q1.2
Q1.3
Q1.4


n = 1e+06, T = 128 : moyenne ± ecart-type (barres), points = mesures
temps (s)
10^{-2}
10^{-3}
10^{0}
10^{1}
10^{2}
10^{3}
pas par thread (S)
Q1.1 (T = 1)
Q1.2
Q1.3
Q1.4


n = 1e+06, T = 256 : moyenne ± ecart-type (barres), points = mesures
temps (s)
10^{-2}
10^{-3}
10^{0}
10^{1}
10^{2}
10^{3}
pas par thread (S)
Q1.1 (T = 1)
Q1.2
Q1.3
Q1.4


n = 1e+08, T = 32 : moyenne ± ecart-type (barres), points = mesures
temps (s)
10^{0}
10^{-1}
10^{0}
10^{1}
10^{2}
10^{3}
pas par thread (S)
Q1.1 (T = 1)
Q1.2
Q1.3
Q1.4



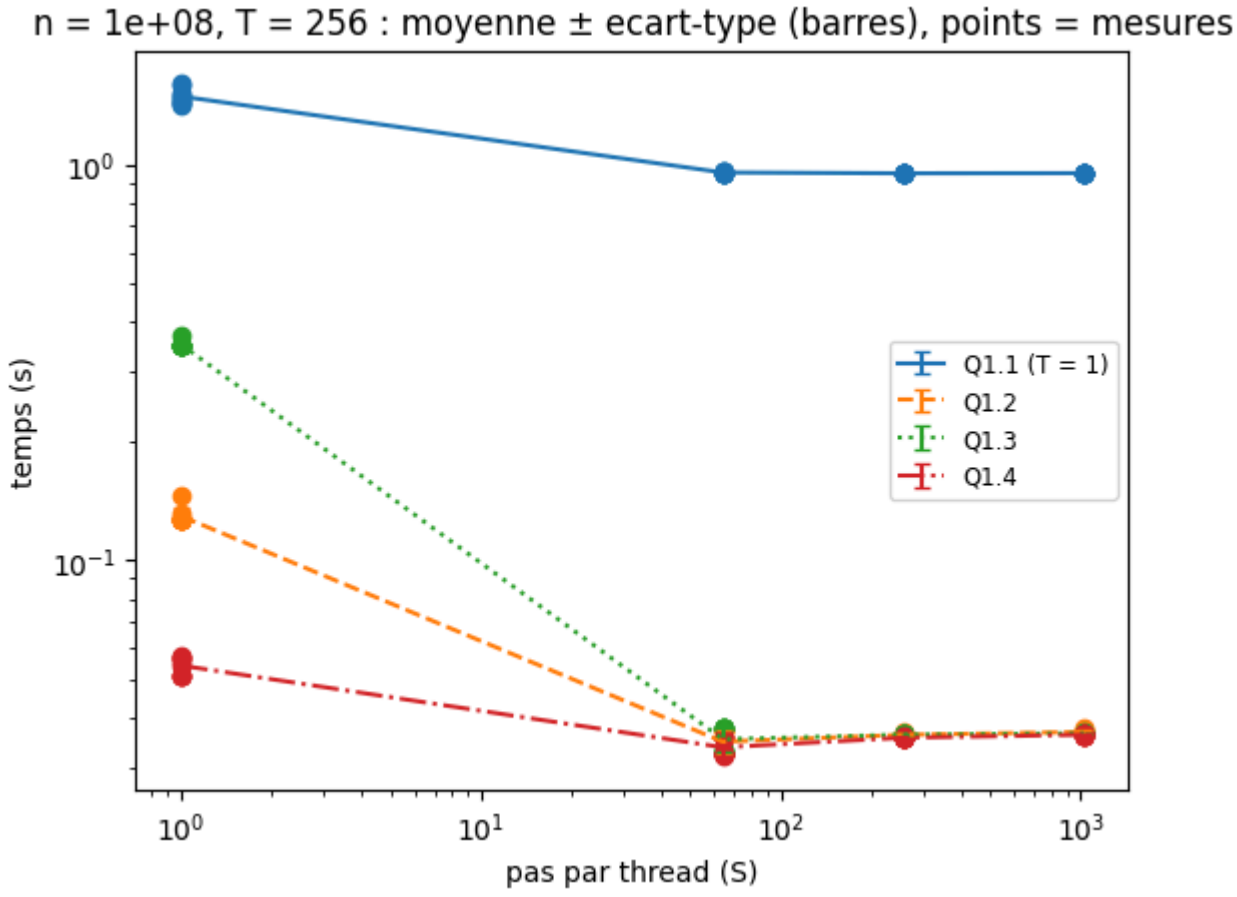


Tableau des meilleures configurations (Q1.4)

pas	meilleure config (T, S)	blocs	temps moyen ± écart-type (s)	exécutions	débit (milliards de pas/s)
1e+06	T = 32, S = 256	123	0.0007 ± 0.0000	10	1.4
1e+08	T = 256, S = 64	6104	0.0337 ± 0.0018	10	3.0
1e+10	T = 32, S = 1024	305176	3.00 ± 0.0014	10	3.3
1e+12	T = 32, S = 1024	30517579	300 ± 0.3748	2	3.3

Question 1.7 - Discussion

Les temps cités sont des moyennes sur 10 exécutions (seulement 2 exécutions à 10^12 pas car beaucoup trop long).

Meilleure configuration de la Q1.4 pour chaque nombre de pas (temps en secondes)

nombre de pas	T	S	blocs	temps moyen ± écart-type (s)	exécutions	débit (milliards de pas/s)
10^6	32	256	123	0.00074 ± 0.00003	10	1.4
10^8	256	64	6104	0.0337 ± 0.0018	10	3.0
10^10	32	1024	305176	2.9974 ± 0.0014	10	3.3
10^12	32	1024	30517579	299.51 ± 0.37	2	3.3

10^6 pas, temps en millisecondes, 10 exécutions par configuration

version	T	S = 1	S = 64	S = 256	S = 1024
Q1.1	1	19.6 ± 2.0	12.09 ± 0.17	11.67 ± 0.77	13.15 ± 0.74
Q1.2	1	20.8 ± 2.0	12.35 ± 0.31	12.26 ± 0.16	13.54 ± 0.41
Q1.2	32	2.65 ± 0.47	1.17 ± 0.13	1.46 ± 0.59	1.31 ± 0.15
Q1.2	64	2.27 ± 0.24	1.25 ± 0.33	1.089 ± 0.097	1.46 ± 0.23
Q1.2	128	2.39 ± 0.34	1.17 ± 0.17	1.151 ± 0.093	1.39 ± 0.13
Q1.2	256	2.47 ± 0.54	1.16 ± 0.36	1.41 ± 0.22	2.09 ± 0.20
Q1.3	1	12.85 ± 0.24	11.97 ± 0.27	12.21 ± 0.17	13.46 ± 0.23
Q1.3	32	2.27 ± 0.62	1.20 ± 0.21	1.14 ± 0.19	1.20 ± 0.12
Q1.3	64	2.07 ± 0.27	1.06 ± 0.12	1.13 ± 0.20	1.30 ± 0.15
Q1.3	128	2.727 ± 0.064	1.33 ± 0.57	1.19 ± 0.17	1.36 ± 0.15
Q1.3	256	4.76 ± 0.14	1.108 ± 0.081	1.40 ± 0.24	1.93 ± 0.13
Q1.4	1	12.16 ± 0.12	11.697 ± 0.098	11.94 ± 0.11	13.075 ± 0.040
Q1.4	32	0.899 ± 0.050	0.851 ± 0.058	0.737 ± 0.025	0.893 ± 0.028
Q1.4	64	0.976 ± 0.083	0.796 ± 0.096	0.804 ± 0.109	1.033 ± 0.043
Q1.4	128	0.944 ± 0.045	0.750 ± 0.058	0.871 ± 0.055	1.044 ± 0.051
Q1.4	256	0.973 ± 0.112	0.824 ± 0.059	1.046 ± 0.051	1.619 ± 0.025

10^8 pas, temps en secondes, 10 exécutions par configuration

version	T	S = 1	S = 64	S = 256	S = 1024
Q1.1	1	1.488 ± 0.066	0.9559 ± 0.0021	0.9513 ± 0.0014	0.9520 ± 0.0014
Q1.2	1	1.498 ± 0.058	0.9667 ± 0.0014	0.9606 ± 0.0011	0.9591 ± 0.0011
Q1.2	32	0.1412 ± 0.0055	0.0353 ± 0.0026	0.03624 ± 0.00012	0.03665 ± 0.00034
Q1.2	64	0.1364 ± 0.0070	0.0346 ± 0.0025	0.03642 ± 0.00042	0.03598 ± 0.00010
Q1.2	128	0.135 ± 0.012	0.0346 ± 0.0025	0.03643 ± 0.00050	0.03625 ± 0.00012
Q1.2	256	0.1295 ± 0.0060	0.0349 ± 0.0025	0.03627 ± 0.00023	0.03689 ± 0.00042
Q1.3	1	1.0380 ± 0.0071	0.9566 ± 0.0011	0.9564 ± 0.0013	0.9567 ± 0.0016
Q1.3	32	0.10894 ± 0.00005	0.0340 ± 0.0025	0.03613 ± 0.00023	0.03650 ± 0.00013
Q1.3	64	0.1202 ± 0.0059	0.0341 ± 0.0025	0.03615 ± 0.00010	0.03611 ± 0.00029
Q1.3	128	0.1840 ± 0.0055	0.0342 ± 0.0025	0.03614 ± 0.00007	0.03622 ± 0.00007
Q1.3	256	0.3513 ± 0.0061	0.0354 ± 0.0026	0.03629 ± 0.00009	0.03660 ± 0.00006
Q1.4	1	1.0142 ± 0.0058	0.95875 ± 0.00085	0.9582 ± 0.0015	0.9590 ± 0.0015
Q1.4	32	0.0447 ± 0.0018	0.03560 ± 0.00022	0.03566 ± 0.00017	0.03612 ± 0.00006
Q1.4	64	0.0521 ± 0.0011	0.03495 ± 0.00062	0.03561 ± 0.00010	0.03572 ± 0.00013
Q1.4	128	0.05472 ± 0.00003	0.03584 ± 0.00028	0.03560 ± 0.00004	0.03590 ± 0.00009
Q1.4	256	0.0543 ± 0.0027	0.0337 ± 0.0018	0.03566 ± 0.00014	0.03629 ± 0.00009

Q1.4 à 10^10 pas, temps en secondes, 10 exécutions par configuration

T	S = 1	S = 64	S = 256	S = 1024
32	4.44 ± 0.18	3.0074 ± 0.0027	3.0007 ± 0.0029	2.9974 ± 0.0014
64	4.4958 ± 0.0011	3.02900 ± 0.00019	3.01225 ± 0.00006	3.00875 ± 0.00009
128	4.67396 ± 0.00069	3.03132 ± 0.00009	3.01288 ± 0.00004	3.00887 ± 0.00006
256	4.85216 ± 0.00052	3.03389 ± 0.00010	3.01343 ± 0.00006	3.00903 ± 0.00007

Q1.4 à 10^12 pas, temps en secondes (nombre d'exécutions entre parenthèses)

T	S = 256	S = 1024
32	300.46 ± 0.48 (2)	299.51 ± 0.37 (2)
64	301.1830 ± 0.0014 (2)	300.804 ± 0.019 (2)
128	301.197 ± 0.042 (2)	300.796 ± 0.015 (2)
256	300.77 ± 0.75 (2)	300.8435 ± 0.0021 (2)

Un thread par bloc est très inefficace. À 10^8 pas, la Q1.1 met 0.95 s (S = 256) alors que la Q1.4 avec 128 threads par bloc met 0.036 s, soit 27 fois moins. Le même écart apparaît dans les versions Q1.2, Q1.3 et Q1.4 dès qu'on les lance avec T = 1 (0.96 s à 10^8 pas), donc la cause n'est pas la méthode de réduction mais le nombre de threads par bloc. Après en avoir discuté avec une LLM, j'ai cru comprendre qu'un GPU exécute les threads par groupes de 32 (les warps) : donc si on se limite à un unique thread par bloc, cela occupe un warp dont 31 voies restent inactives, ce qui limite l'utilisation à environ 1/32. Le facteur observé (27) est assez cohérent avec cette limite.

Au-delà d'un warp, le nombre de threads par bloc ne change plus rien. Pour S >= 64, tous les T entre 32 et 256 donnent le même temps : entre 0.034 et 0.036 s à 10^8 pas, et entre 3.00 et 3.03 s à 10^10 pas (Q1.4). Le GPU semble saturé et le temps est celui du calcul. Une nuance : aux grandes tailles, T = 32 est un peu plus rapide que les autres, de 0.011 à 0.027 s sur 3 s à 10^10 pas (0.4 à 0.9 %) et de 1.3 s sur 300 s à 10^12 pas avec S = 1024 (0.4 %), un écart faible mais supérieur aux écarts-types de ces configurations, et je ne l'explique pas.

On remarque aussi que le nombre de pas par thread compte surtout aux extrêmes. Avec S = 1, chaque thread ne calcule qu'un pas : à 10^10 pas, il y a de 3.9 x 10^7 blocs (T = 256) à 3.1 x 10^8 blocs (T = 32), et chacun produit un total qu'il faut ensuite réduire. Le temps passe alors à 4.44-4.85 s au lieu de 3.0 s, soit 48 à 62 % de plus, et il augmente avec T (4.44 s à T = 32, 4.85 s à T = 256). Mon hypothèse est que le calcul par pas est si court que le coût de la réduction (écriture et relecture des totaux, avec arbre de log2(T) étapes) devient comparable au calcul (si le temps l'avait permis, cela aurait pu être mesuré séparément pour confirmer ou infirmer l'hypothèse). À T = 32 et S = 1, les deux tableaux de totaux font 2 x 2.5 Go, plus que les 4 Go de la carte. L'exécution réussit quand même et donne pi correct, possiblement grâce à un repli du pilote sur la mémoire du système, et j'ai été très surpris de ne pas voir de ralentissement lié à cela, puisque T = 32 est plus rapide que T = 64. À l'inverse, trop peu de threads pénalise les petites tailles : à 10^6 pas avec S = 1024 et T = 256, il n'y a qu'environ 1000 threads (4 blocs) pour 14 multiprocesseurs, et la Q1.4 met 1.6 ms contre 0.7 à 0.9 ms pour les autres combinaisons.

Pour S >= 64, les versions Q1.2, Q1.3 et Q1.4 sont indiscernables (0.034 à 0.037 s à 10^8 pas) : la réduction ne pèse presque rien devant le calcul. Les différences apparaissent quand il y a beaucoup de réduction à faire, par exemple avec S = 1 à 10^8 pas. La Q1.4 est alors la plus rapide et la plus stable (0.045 à 0.055 s selon T), car l'arbre en mémoire partagée ne demande que log2(T) étapes parallèles. La Q1.2 est 2.5 à 3 fois plus lente (0.13 à 0.14 s), car un seul thread par bloc additionne les T valeurs, l'une après l'autre, en mémoire globale. La Q1.3 se dégrade quand T augmente : 0.109 s à T = 32, 0.184 s à T = 128 et 0.351 s à T = 256. Je suppose que cela est dû au fait que les T threads d'un bloc font leur `atomicAdd` sur la même variable partagée, donc l'un après l'autre (cette sérialisation qui s'aggrave avec le nombre de threads avait aussi été vue avec `atomic` dans le TP OpenMP). À 10^6 pas, la Q1.4 reste la meilleure (environ 0.8 ms à T >= 32 contre 1.1 à 1.3 ms pour Q1.2 et Q1.3).

Le meilleur débit passe de 1.4 milliard de pas par seconde à 10^6 pas, à 3.0 à 10^8, puis 3.3 à 10^10 et à 10^12. À 10^6 pas, le temps (0.7 ms) semble dominé par les coûts fixes (lancement de plusieurs kernels, copie du résultat vers le CPU) qui ne sont pas amortis. À partir de 10^10 pas, le débit est constant et le temps est proportionnel au nombre de pas : 301 s à 10^12 contre 3.0 s à 10^10, soit 100 fois plus pour 100 fois plus de pas. Les écarts-types sont très faibles (de 0.001 s à 0.75 s sur environ 300 s à 10^12 pas, soit au plus 0.25 %) donc les mesures semblent stables.

Le code séquentiel de départ (compilé avec `-O2`) met en moyenne 0.158 s (écart-type 0.002 s) à 10^8 pas sur 10 exécutions, contre 0.034 s pour le meilleur kernel, soit environ 5 fois moins. À 10^10 pas en revanche, le TP OpenMP obtenait environ 3.1 s avec 8 threads, contre 3.0 s sur le GPU, donc surprenamment mon GPU n'est pas plus rapide que mon processeur parallélisé (peut-être que la GTX est limitée à cause du calcul en double précision ? Il aurait fallu faire d'autres mesures pour faire une hypothèse sérieuse, mais par manque de temps on ne développera pas davantage ce point).

Conclusion : sur ce GPU, le facteur déterminant est d'avoir assez de threads actifs par bloc (au moins un warp de 32, et cela peut permettre de gagner un facteur 27). Une fois cette condition remplie, le nombre de threads par bloc et le nombre de pas par thread ne changent presque plus le temps, sauf à S = 1 où la réduction devient coûteuse. Dans ce cas, la réduction en arbre (Q1.4) est la plus efficace et la plus stable, alors que les atomiques (Q1.3) et la somme par un seul thread (Q1.2) se dégradent.

2. Matrix-Vector operation

Question 2.1 - Code séquentiel

Code complété (le même qu'au TP OpenMP, avec l'écriture des mesures dans le fichier CSV commun) :

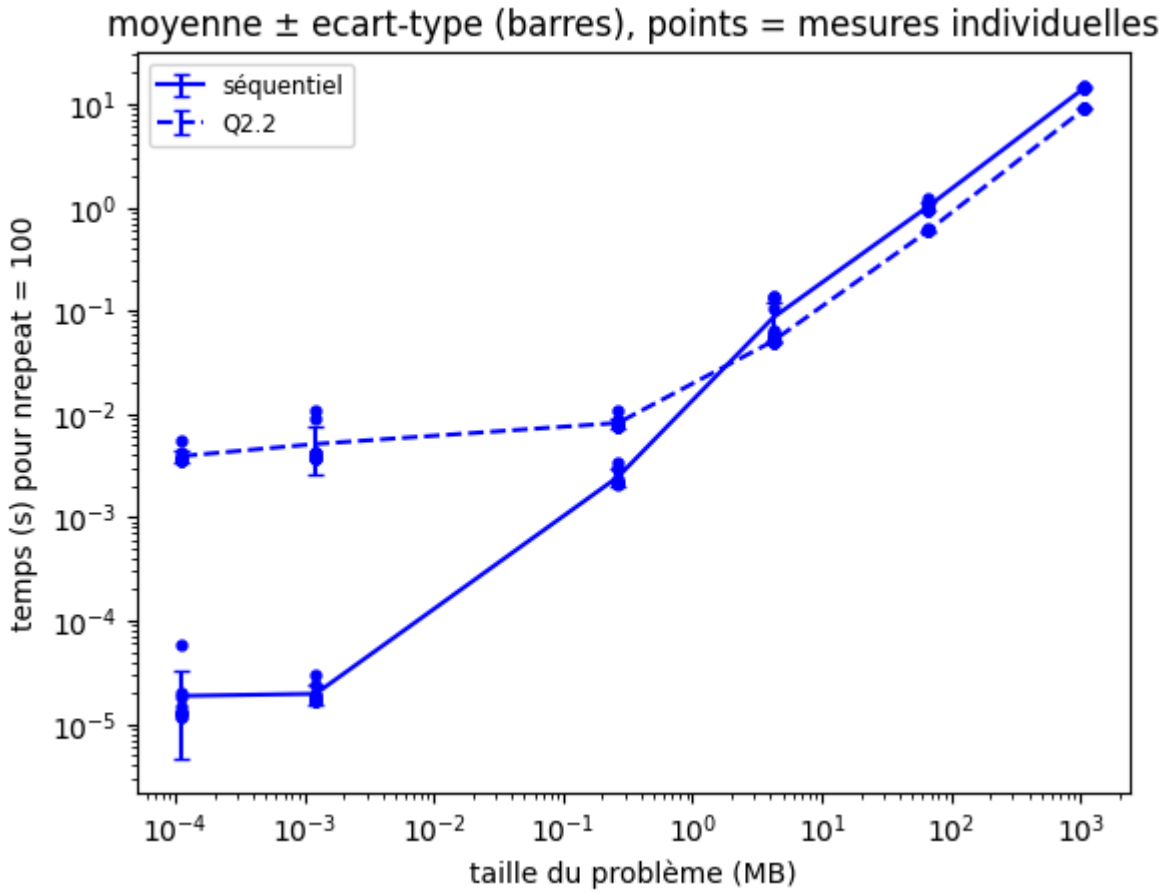
Question 2.2 - Port CUDA (un thread par bloc)

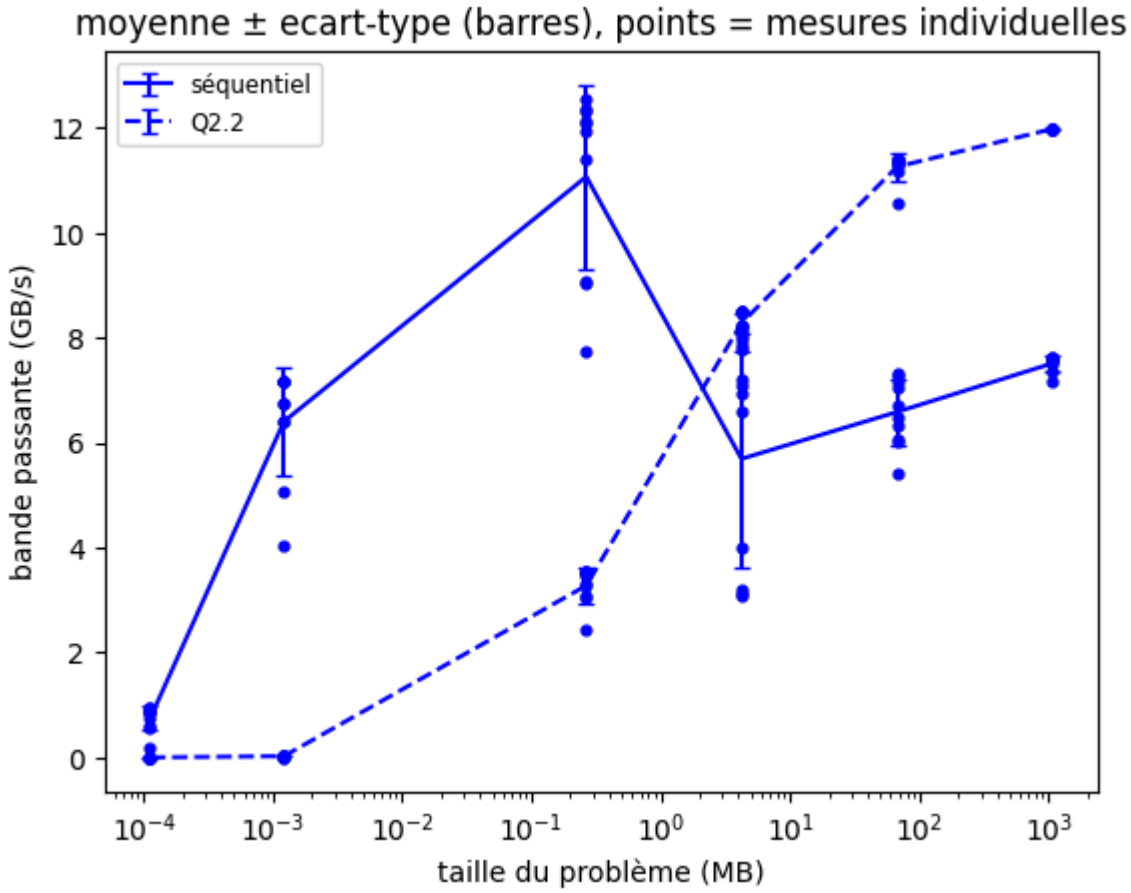
Le kernel est lancé avec `N` blocs d'un seul thread, un bloc par ligne de `A`. Le bloc `i` calcule la somme des produits de la ligne `i` avec `x`, la multiplie par `y[i]` et écrit le résultat dans un tableau de `N` valeurs. Ce tableau est copié vers le CPU à chaque répétition et la somme finale est faite sur le CPU. Les copies de `A`, `x` et `y` vers le GPU sont faites une seule fois avant le chronomètre, comme l'initialisation dans la version OpenMP. Le temps mesuré comprend donc les `nrepeat` = 100 répétitions du kernel et de la copie du résultat, mais pas les copies initiales (pas pertinent car constant).

Question 2.3 - Évaluation de performance (un thread par bloc)

Les tailles sont les paires (N, M) de l'énoncé, qui sont des exposants de 2 : (2, 1), (4, 3), (8, 7), (10, 9), (12, 11) et (14, 13). La dernière paire (16, 15) est exclue pour la même raison que celle détaillée dans le TP OpenMP précédent : N x M = 2^31 dépasse la capacité d'un `int`, et la matrice ferait de toute façon 16 Go, bien plus que les 4 Go de la carte. Le code séquentiel est mesuré sur la même grille pour servir de référence. Chaque exécution fait 100 répétitions du produit (`nrepeat`) et écrit une ligne avec le temps et la bande passante, ajoutée à la feuille `partie2_vecteur` du classeur.

Question 2.4 - Figures de performance (un thread par bloc)





Question 2.5 - Discussion (un thread par bloc)

Séquentiel et Q2.2, nrepeat = 100, 10 exécutions par taille

N x M (taille de A)	séquentiel : temps (s)	Q2.2 : temps (s)	séquentiel : bande passante (GB/s)	Q2.2 : bande passante (GB/s)
2^2 x 2^1 (112 o)	0.00002 ± 0.00001	0.00391 ± 0.00058	0.743 ± 0.231	0.00291 ± 0.00033
2^4 x 2^3 (1216 o)	0.00002 ± 0.00000	0.00512 ± 0.00253	6.40 ± 1.04	0.0273 ± 0.0080
2^8 x 2^7 (265 Ko)	0.00246 ± 0.00046	0.00819 ± 0.00103	11.1 ± 1.8	3.28 ± 0.35
2^10 x 2^9 (4.21 Mo)	0.0857 ± 0.0366	0.0508 ± 0.0012	5.69 ± 2.06	8.28 ± 0.20
2^12 x 2^11 (67.2 Mo)	1.03 ± 0.11	0.597 ± 0.014	6.59 ± 0.64	11.26 ± 0.26
2^14 x 2^13 (1.07 Go)	14.30 ± 0.29	8.9646 ± 0.0058	7.51 ± 0.15	11.9797 ± 0.0078

Pour les très petites matrices, le CPU est beaucoup plus rapide : 0.00002 s pour 100 répétitions à 2^2 x 2^1 contre 0.0039 s sur le GPU. À chaque répétition, le GPU doit lancer un kernel, copier le résultat vers le CPU et se synchroniser, ce qui semble coûter environ 40 à 50 microsecondes (d'après nos mesures) et c'est bien plus que le calcul lui-même. Le GPU passe devant à partir de 4.21 Mo (0.051 s contre 0.086 s) et l'écart augmente ensuite : à 1.07 Go, la Q2.2 met 8.96 s contre 14.3 s pour le séquentiel, soit 1.6 fois moins. Le CPU atteint 11.1 GB/s à 265 Ko, où la matrice tient dans ses caches, puis seulement 5.7 à 7.5 GB/s au-delà.

Ce gain reste faible. À 1.07 Go, la Q2.2 atteint 11.98 GB/s. D'après les propriétés de la carte lues avec CUDA (mémoire à 6001 MHz sur un bus de 128 bits), la bande passante mémoire théorique est de 192 GB/s, soit seulement 6 %. Je pense que c'est parce que chaque bloc n'a qu'un thread : il occupe un warp dont une seule voie travaille, et ce thread lit sa ligne élément par élément sans que d'autres threads l'aident à masquer la latence de la mémoire. C'est une hypothèse qu'il aurait fallu vérifier en mesurant d'autres configurations de cette version. Les mesures sont très stables à grande taille (écart-type de 0.006 s sur 8.96 s à 1.07 Go) et plus bruitées pour les petites matrices, où le temps est dominé par les lancements.

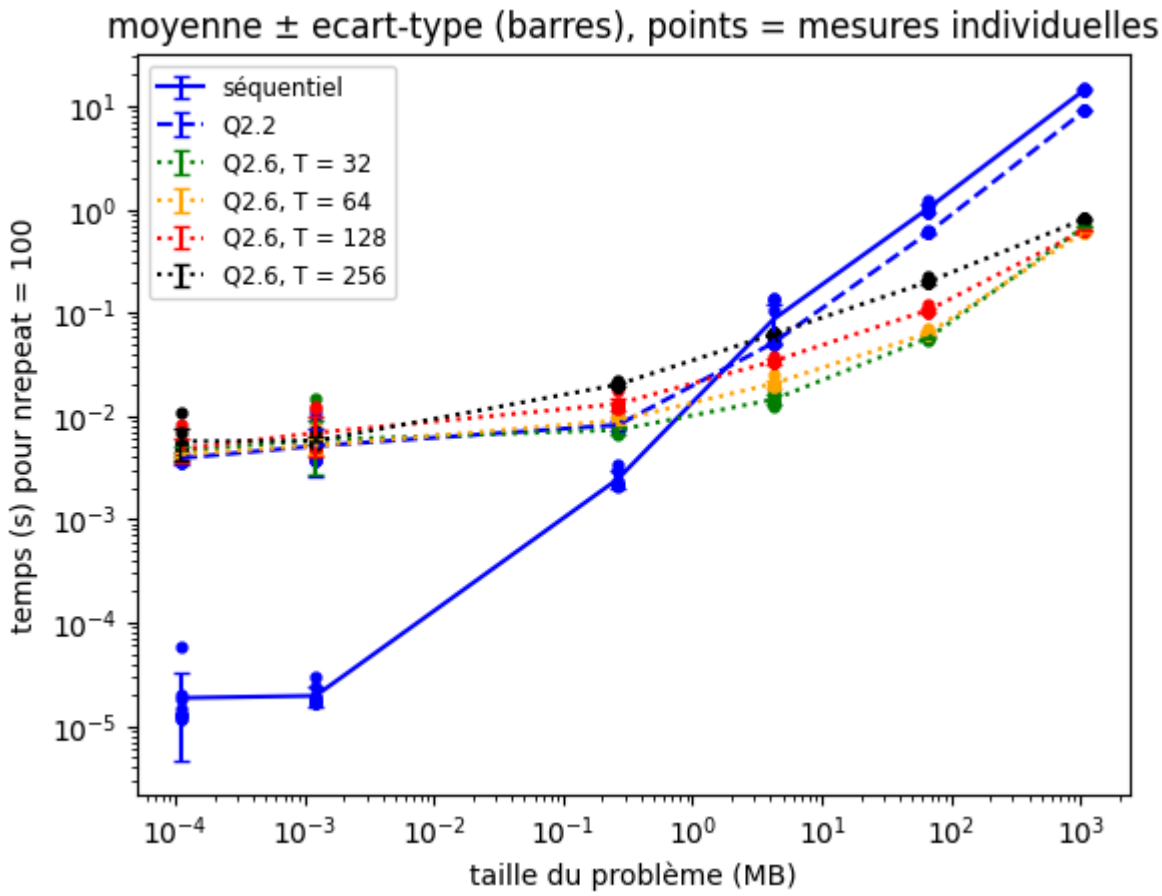
Question 2.6 - Plusieurs threads par bloc et atomiques

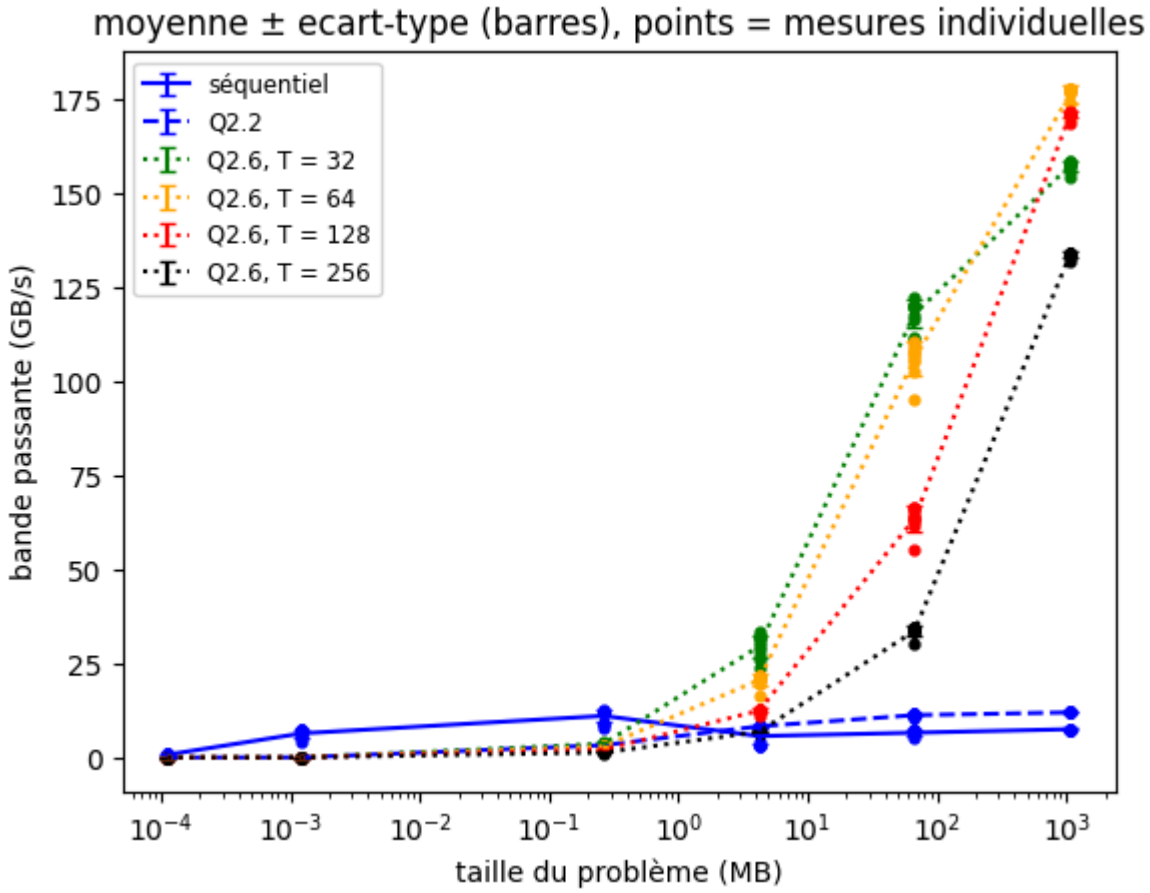
Le bloc `i` traite toujours la ligne `i`, mais avec `T` threads (argument `-T`). Le thread `t` prend les colonnes `t`, `t + T`, `t + 2T`, etc. de la ligne, ce qui fait que des threads voisins lisent des cases voisines de `A`. Chaque thread multiplie sa somme partielle par `y[i]` et l'ajoute au résultat global avec `atomicAdd`. Le résultat est remis à zéro (`cudaMemset`) avant chaque répétition, et il n'y a plus de tableau intermédiaire ni de somme sur le CPU.

Question 2.7 - Évaluation de performance (plusieurs threads par bloc)

Mêmes paires (N, M) que pour la question 2.3. L'énoncé ne fixe pas le nombre de threads par bloc : je prends T = 32, 64, 128 et 256, soit un warp complet jusqu'à une taille de bloc courante.

Question 2.8 - Figures de performance (plusieurs threads par bloc)





Question 2.9 - Discussion (plusieurs threads par bloc)

Q2.6, temps en secondes, nrepeat = 100, 10 exécutions par configuration

N x M (taille de A)	T = 32	T = 64	T = 128	T = 256
2^2 x 2^1 (112 o)	0.00459 ± 0.00049	0.00425 ± 0.00011	0.00480 ± 0.00124	0.00568 ± 0.00200
2^4 x 2^3 (1216 o)	0.00586 ± 0.00318	0.00516 ± 0.00096	0.00686 ± 0.00283	0.00573 ± 0.00053
2^8 x 2^7 (265 Ko)	0.00731 ± 0.00069	0.00905 ± 0.00068	0.0131 ± 0.0018	0.0201 ± 0.0010
2^10 x 2^9 (4.21 Mo)	0.0144 ± 0.0016	0.0205 ± 0.0019	0.0338 ± 0.0017	0.0618 ± 0.0017
2^12 x 2^11 (67.2 Mo)	0.0570 ± 0.0019	0.0634 ± 0.0028	0.1061 ± 0.0061	0.1997 ± 0.0079
2^14 x 2^13 (1.07 Go)	0.6826 ± 0.0056	0.6094 ± 0.0091	0.6286 ± 0.0028	0.8031 ± 0.0039

Q2.6, bande passante en GB/s, nrepeat = 100, 10 exécutions par configuration

N x M (taille de A)	T = 32	T = 64	T = 128	T = 256
2^2 x 2^1 (112 o)	0.00246 ± 0.00024	0.00264 ± 0.00007	0.00242 ± 0.00038	0.00211 ± 0.00046
2^4 x 2^3 (1216 o)	0.0235 ± 0.0057	0.0241 ± 0.0032	0.0198 ± 0.0059	0.0214 ± 0.0017
2^8 x 2^7 (265 Ko)	3.65 ± 0.32	2.94 ± 0.20	2.06 ± 0.23	1.319 ± 0.065
2^10 x 2^9 (4.21 Mo)	29.5 ± 3.1	20.6 ± 1.6	12.45 ± 0.57	6.81 ± 0.18
2^12 x 2^11 (67.2 Mo)	117.9 ± 3.7	106.1 ± 4.4	63.4 ± 3.3	33.7 ± 1.2
2^14 x 2^13 (1.07 Go)	157.3 ± 1.3	176.3 ± 2.6	170.84 ± 0.76	133.72 ± 0.65

Ajouter des threads dans le bloc change complètement le résultat à grande taille. À 1.07 Go, le meilleur temps est 0.609 s avec T = 64, soit 14.7 fois moins que la Q2.2 (8.96 s) et 23 fois moins que le séquentiel (14.3 s). La bande passante atteint 176 GB/s, soit 92 % du maximum théorique de la carte (192 GB/s). À cette taille le calcul est limité par la mémoire, et il reste peu à gagner.

Pour les tailles petites et moyennes, le temps augmente avec T, ce qui est l'inverse de ce à quoi je m'attendais. À 4.21 Mo, il passe de 0.0144 s (T = 32) à 0.0618 s (T = 256), et à 67.2 Mo de 0.0570 s à 0.1997 s. Chaque thread fait un atomicAdd sur le résultat global, donc il y a N x T additions atomiques par répétition, toutes sur la même adresse. Si on divise ce nombre par le temps mesuré, on trouve environ 5.2 x 10^8 additions atomiques par seconde à T = 256, à 67.2 Mo comme à 1.07 Go (4096 x 256 x 100 additions en 0.1997 s, et 16384 x 256 x 100 en 0.8031 s), et environ 4.9 x 10^8 à T = 128 à 67.2 Mo. Ce débit est le même pour deux tailles différentes, ce qui laisse penser que le fait que les additions sont exécutées l'une après l'autre sur une seule adresse limite le temps quand T est grand (environ une addition tous les 3 cycles de l'horloge à 1.68 GHz). C'est mon interprétation et je ne l'ai pas vérifiée avec un autre outil. À T = 32, il y a 8 fois moins d'additions et le temps se rapproche de celui de la lecture de la mémoire (117.9 GB/s à 67.2 Mo).

Aux très petites tailles (112 o et 1216 o), tous les T donnent environ 4 à 7 ms pour 100 répétitions, comme la Q2.2. Le coût d'un lancement de kernel et d'une copie domine, et le nombre de threads ne compte pas. Les mesures sont bruitées à ces tailles, avec un écart-type parfois du même ordre que la moyenne (0.00318 pour 0.00586 s à 2^4 x 2^3 et T = 32).

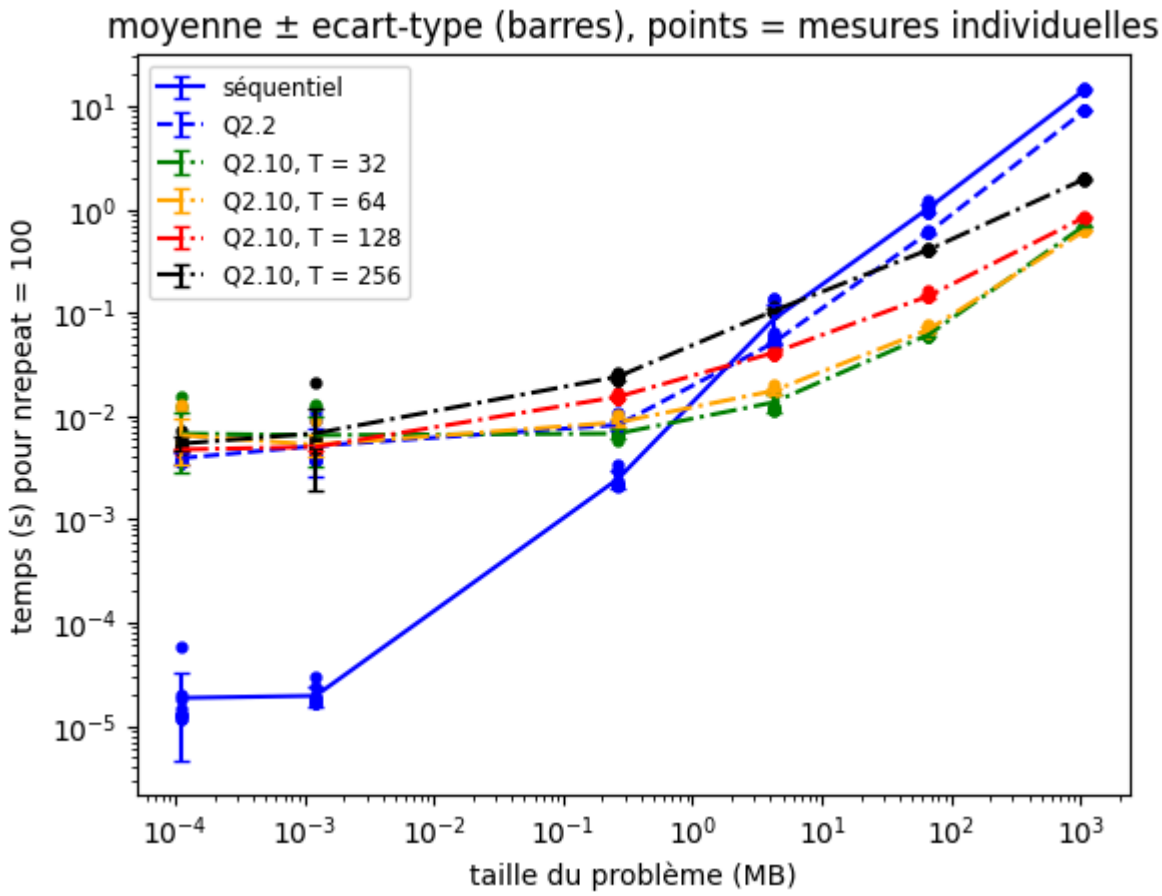
Question 2.10 - Mémoire partagée et atomiques

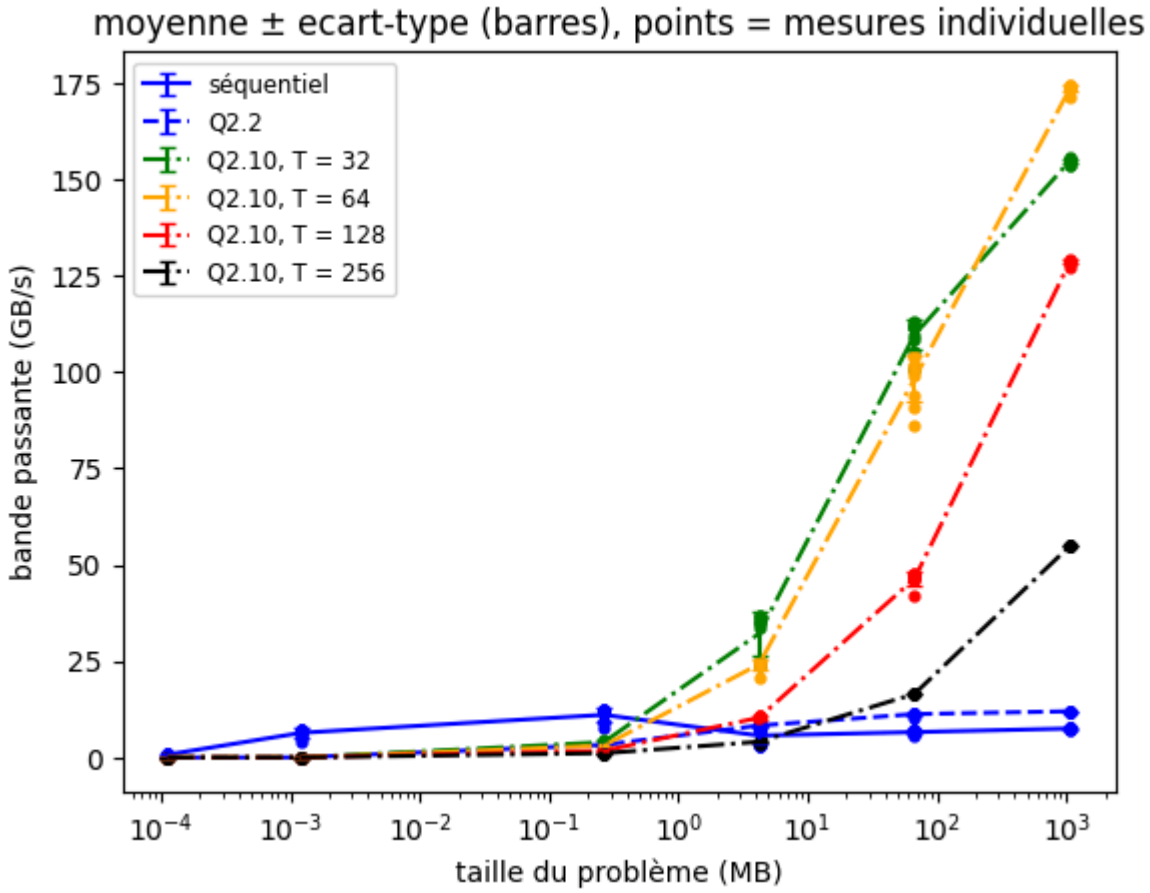
Chaque bloc a une variable `__shared__` qui contient le total de sa ligne. Les `T` threads y ajoutent leur somme partielle avec `atomicAdd` (en mémoire partagée), puis un seul thread par bloc ajoute le total du bloc, multiplié par `y[i]`, au résultat global avec un `atomicAdd`. Il y a donc `N` additions atomiques globales au lieu de `N x T` à la question 2.6. Deux `__syncthreads()` garantissent que le total est initialisé avant les ajouts et est complet avant la lecture.

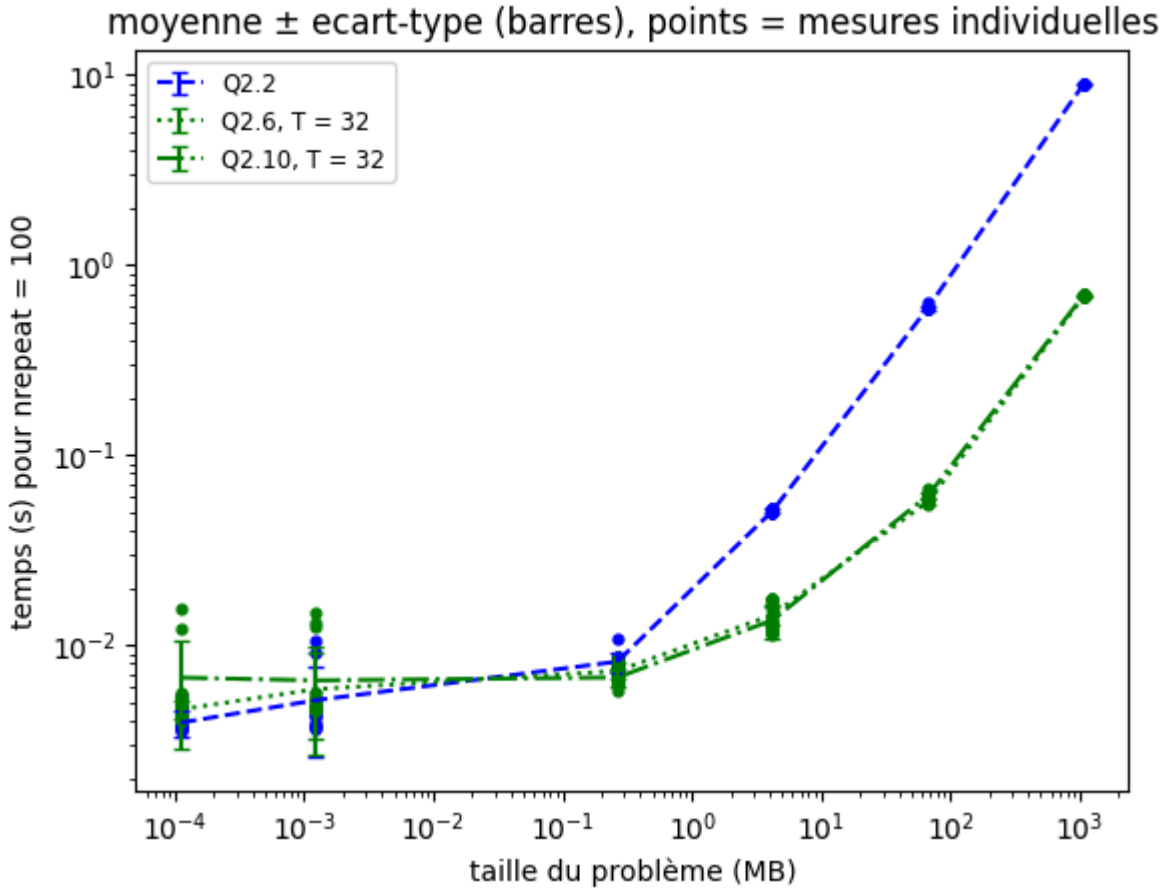
Question 2.11 - Évaluation de performance (mémoire partagée)

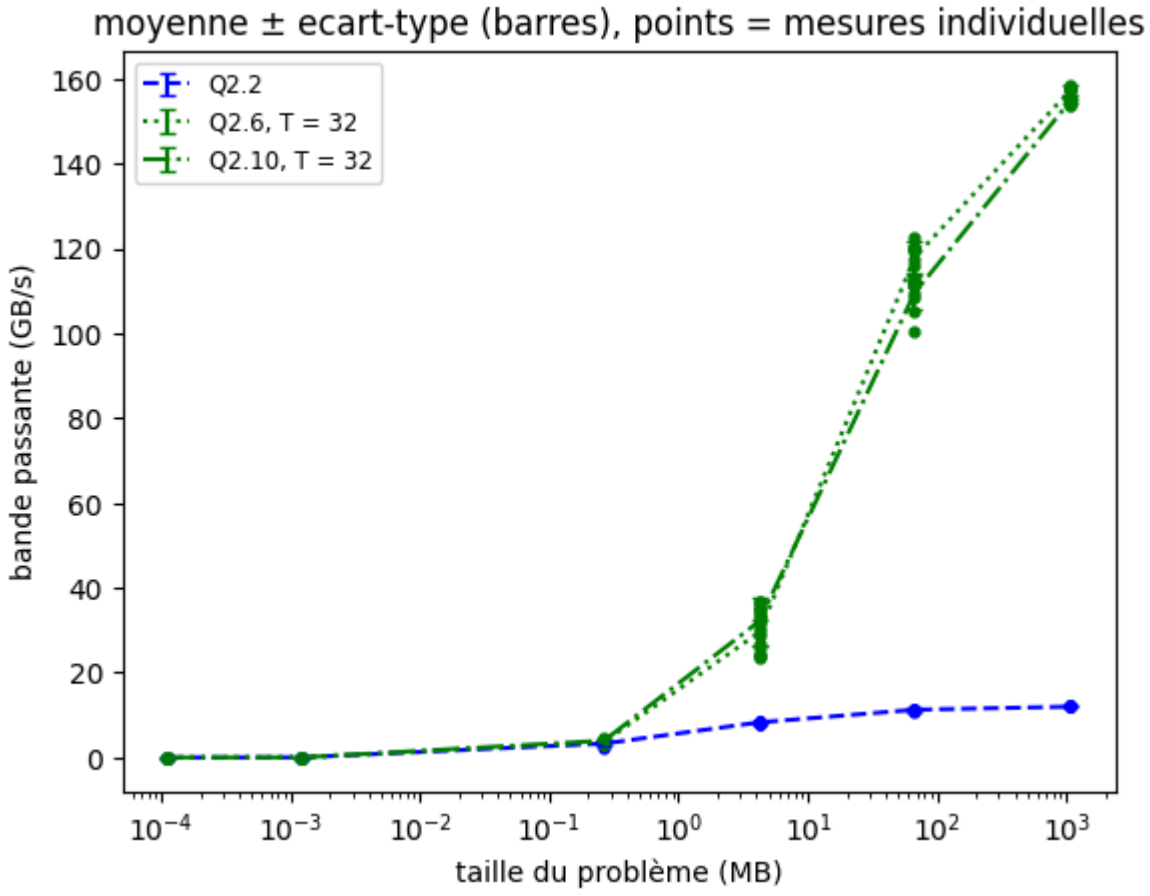
Mêmes paires (N, M) et mêmes valeurs de T que pour la question 2.7.

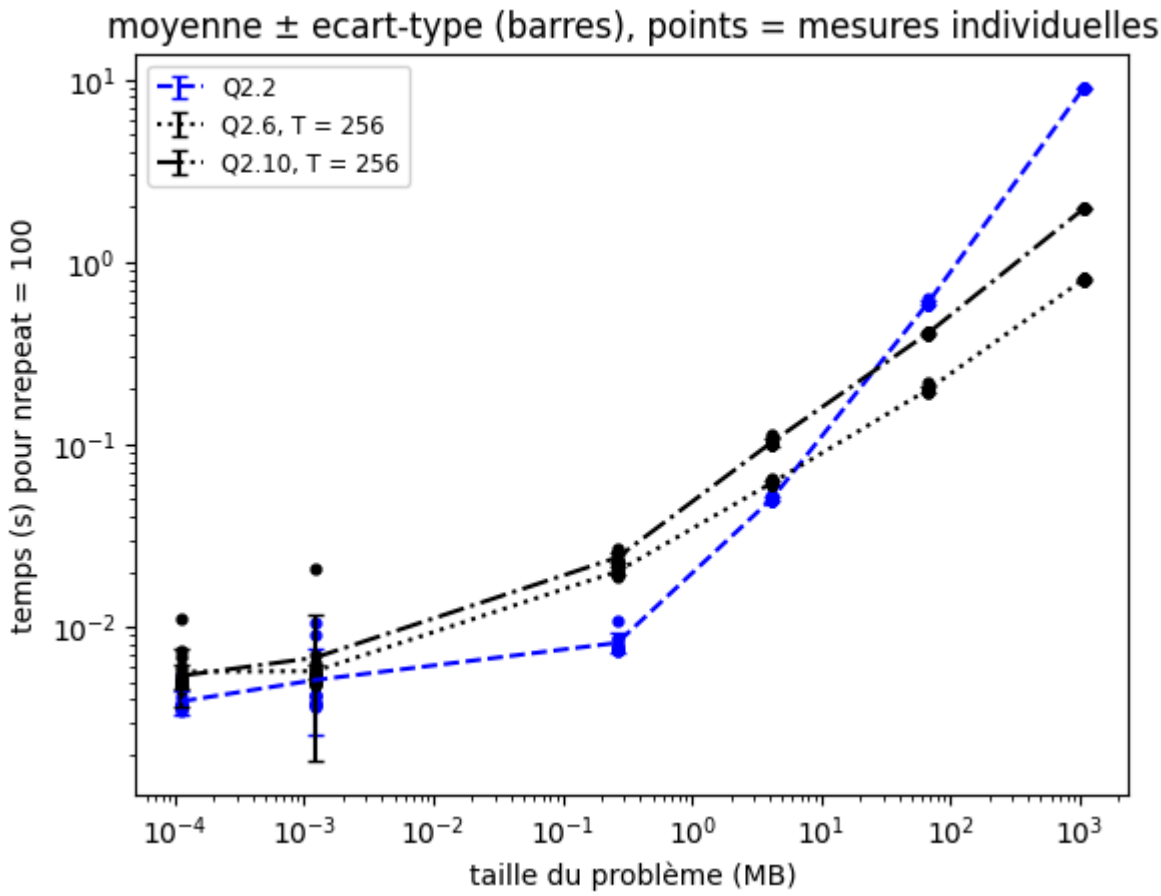
Question 2.12 - Figures de performance (mémoire partagée)

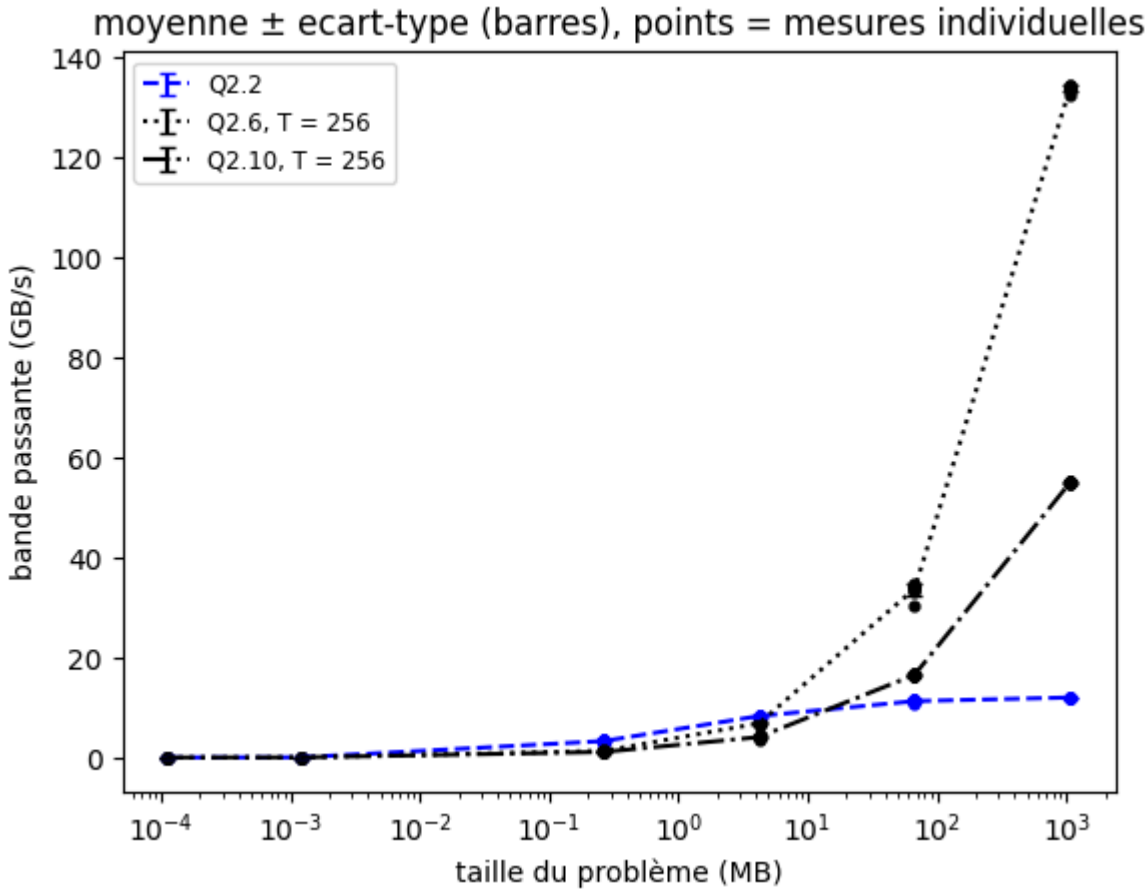





moyenne ± ecart-type (barres), points = mesures individuelles
Q2.2
Q2.6, T = 32
Q2.10, T = 32
temps (s) pour nrepeat = 100
taille du problème (MB)


moyenne ± ecart-type (barres), points = mesures individuelles
Q2.2
Q2.6, T = 32
Q2.10, T = 32
bande passante (GB/s)
taille du problème (MB)


moyenne ± ecart-type (barres), points = mesures individuelles
Q2.2
Q2.6, T = 256
Q2.10, T = 256
temps (s) pour nrepeat = 100
taille du problème (MB)


moyenne ± ecart-type (barres), points = mesures individuelles
Q2.2
Q2.6, T = 256
Q2.10, T = 256
bande passante (GB/s)
taille du problème (MB)

Question 2.13 - Discussion (mémoire partagée)

Q2.10, temps en secondes, nrepeat = 100, 10 exécutions par configuration

N x M (taille de A)	T = 32	T = 64	T = 128	T = 256
2^2 x 2^1 (112 o)	0.00674 ± 0.00390	0.00651 ± 0.00311	0.00474 ± 0.00014	0.00541 ± 0.00083
2^4 x 2^3 (1216 o)	0.00652 ± 0.00331	0.00526 ± 0.00129	0.00502 ± 0.00039	0.00679 ± 0.00493
2^8 x 2^7 (265 Ko)	0.00676 ± 0.00078	0.00868 ± 0.00071	0.01532 ± 0.00081	0.0240 ± 0.0017
2^10 x 2^9 (4.21 Mo)	0.0135 ± 0.0027	0.0175 ± 0.0011	0.0408 ± 0.0012	0.1036 ± 0.0051
2^12 x 2^11 (67.2 Mo)	0.0613 ± 0.0023	0.0686 ± 0.0045	0.1450 ± 0.0058	0.4070 ± 0.0026
2^14 x 2^13 (1.07 Go)	0.6935 ± 0.0026	0.6186 ± 0.0032	0.8349 ± 0.0031	1.9553 ± 0.0035

Rapport du temps Q2.10 / Q2.6 (supérieur à 1 : la Q2.10 est plus lente)

N x M (taille de A)	T = 32	T = 64	T = 128	T = 256
2^2 x 2^1 (112 o)	1.47	1.53	0.99	0.95
2^4 x 2^3 (1216 o)	1.11	1.02	0.73	1.19
2^8 x 2^7 (265 Ko)	0.92	0.96	1.17	1.19
2^10 x 2^9 (4.21 Mo)	0.94	0.85	1.21	1.68
2^12 x 2^11 (67.2 Mo)	1.08	1.08	1.37	2.04
2^14 x 2^13 (1.07 Go)	1.02	1.02	1.33	2.43

La Q2.10 n'améliore pas la Q2.6, contrairement à ce à quoi je m'attendais. Le rapport des temps est proche de 1 pour T = 32 et T = 64 (de 0.85 à 1.08 entre 4.21 Mo et 1.07 Go) et devient supérieur à 1 pour T = 128 et T = 256. À 1.07 Go, la Q2.10 est 1.33 fois plus lente à T = 128 et 2.43 fois plus lente à T = 256 (1.9553 s contre 0.8031 s). À 4.21 Mo elle gagne un peu pour T = 32 et T = 64 (rapports de 0.94 et 0.85), mais l'écart ne dépasse pas 3 ms.

L'idée de la Q2.10 était de remplacer les N x T additions atomiques globales par N additions globales, plus des additions atomiques en mémoire partagée entre les T threads d'un même bloc. Ça supprime le goulot repéré pour la Q2.6, mais la contention se déplace à l'intérieur de chaque bloc, puisque les T threads additionnent sur la même variable partagée. Hypothèse : l'addition atomique d'un double en mémoire partagée serait émulée sur cette carte par une boucle de comparaison et d'échange (compare-and-swap). Ça coûterait cher quand T threads se disputent la même variable et expliquerait que le temps augmente avec T (de 0.6935 s à 1.9553 s entre T = 32 et T = 256 à 1.07 Go). Une réduction en arbre dans la mémoire partagée, comme en partie 1 (Q1.4), éviterait toute addition atomique entre les threads et serait probablement meilleure, mais l'énoncé ne la demande pas.

À T = 32 et T = 64, le temps à 1.07 Go est identique à celui de la Q2.6 (rapport de 1.02), proche du maximum de la mémoire. Quand les additions atomiques ne limitent pas le temps, les deux versions semblent limitées par la mémoire et se valent.

3. Matrix multiplication and floating point precision

Question 3.1 - Port CUDA (un thread par bloc)

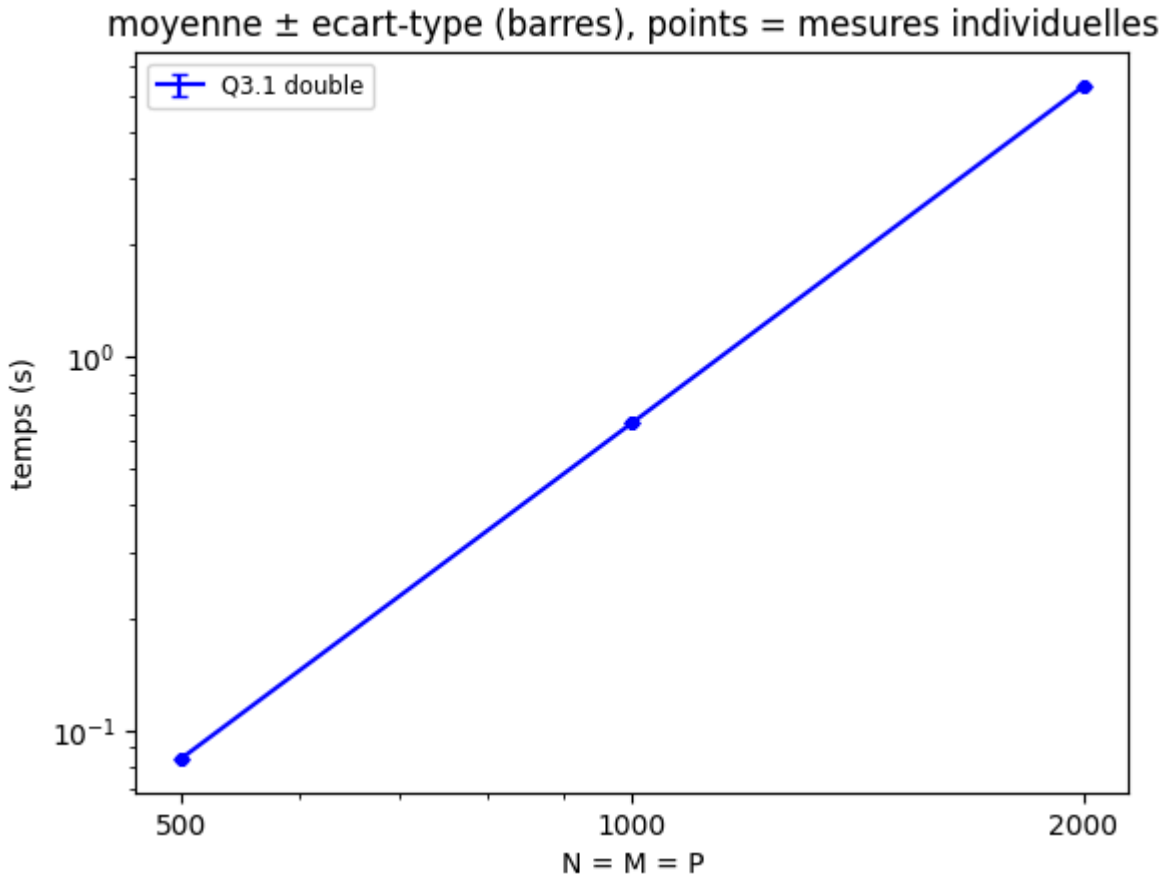
Le kernel est lancé avec un bloc d'un seul thread par élément de `C`, soit une grille de M x N blocs. Le bloc (j, i) calcule `C[i][j]` avec la boucle sur k. Les pas d'indexation sont corrigés par rapport au code de départ (`A` a P colonnes, `B` et `C` en ont M), ce qui ne change rien pour des matrices carrées. Les copies de `A` et `B` vers le GPU et de `C` vers le CPU ne sont pas chronométrées : le temps comprend le lancement du kernel et son exécution (`cudaDeviceSynchronize`). Les MFlops sont calculés avec 2 x N x M x P opérations.

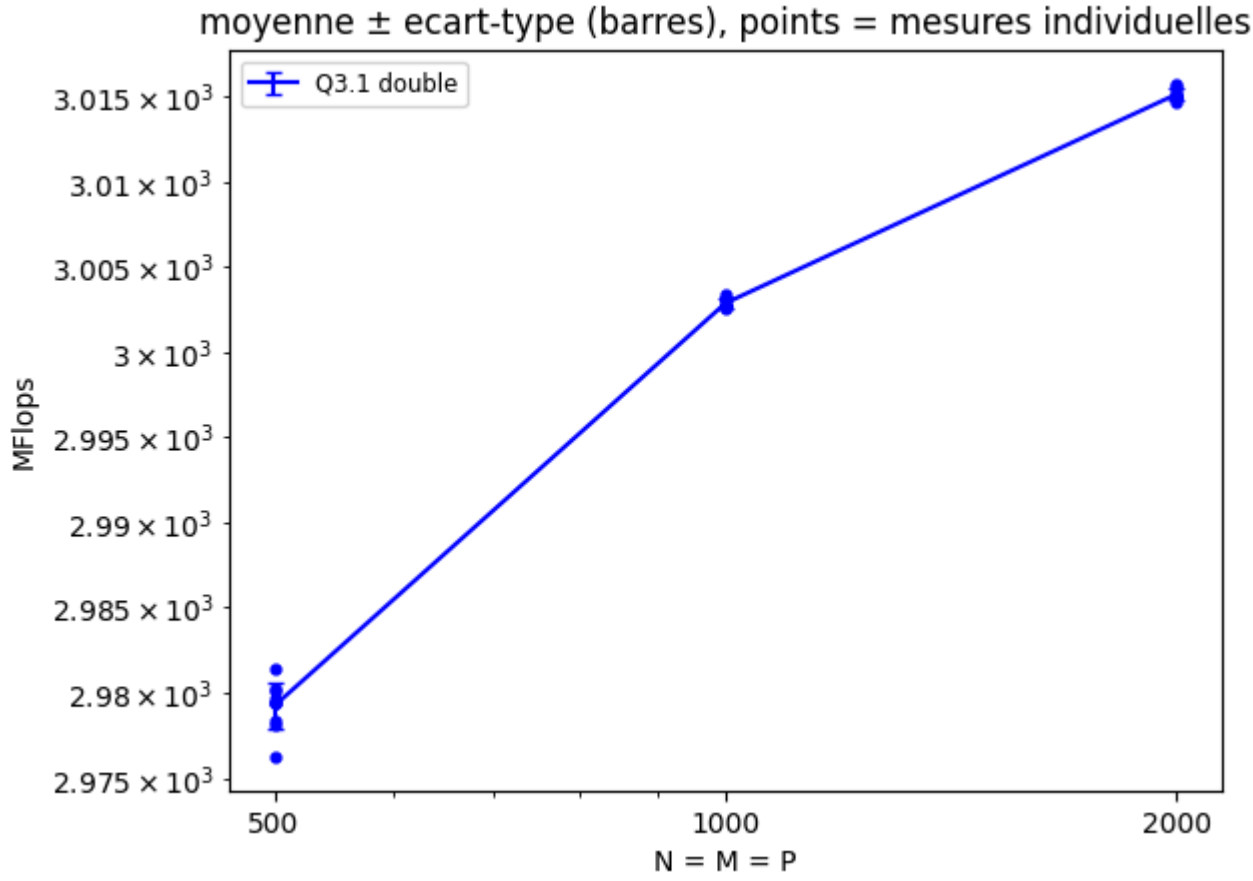
Question 3.2 - Évaluation de performance (un thread par bloc)

L'énoncé demande N et M de 1000 à 18000. Les matrices sont carrées, donc je donne la même valeur à N, M et P, comme au TP OpenMP. Avec 4 Go de mémoire, trois matrices de 18000 x 18000 en `double` (environ 7.8 Go) ne tiennent pas : le programme s'arrête avec « Erreur : memoire insuffisante ». Trois matrices de 12000 x 12000 (3.5 Go) tiennent, mais une seule exécution de la version à plusieurs threads par bloc a pris 4669 s (78 minutes), bien plus que ce que le coût en N^3 laissait prévoir à partir de N = 4000 (11 s) (je pense que c'est parce que la carte est presque pleine). À 4000, la version à un thread par bloc met 47 s. Avec 10 répétitions par configuration, la grille complète n'est donc pas réalisable, et le choix est de travailler avec N = M = P parmi 500, 1000 et 2000.

N = M = P	threads par bloc	répétitions
500, 1000, 2000	1	10

Question 3.3 - Figures de performance (un thread par bloc)





Question 3.4 - Discussion (un thread par bloc)

Q3.1 (un thread par bloc, double), 10 exécutions par taille

N = M = P	temps moyen ± écart-type (s)	MFlops moyens ± écart-type
500	0.08391 ± 0.00004	2979.3 ± 1.4
1000	0.66603 ± 0.00006	3002.85 ± 0.26
2000	5.30651 ± 0.00058	3015.16 ± 0.33

Le temps de la version à un thread par bloc croît comme N^3 : il est multiplié par 7.9 de 500 à 1000 (0.0839 s à 0.666 s) et par 8.0 de 1000 à 2000 (0.666 s à 5.31 s), ce qui correspond à un nombre d'opérations multiplié par 8 à chaque fois. Le débit est presque constant (de 2979 à 3015 MFlops, soit environ 3 GFlops) et les écarts-types sont minuscules (0.0006 s sur 5.31 s à N = 2000). Donc je suppose qu'aucun effet de cache ou de taille n'est visible sur ces trois tailles.

Ce débit de 3 GFlops est très inférieur à ce que la carte peut faire d'après ses spécifications, de l'ordre de 3 TFlops en simple précision et 1/32 de cela en double précision (sans entrer dans le détail de ce calcul). Comme en partie 2, un bloc d'un seul thread n'utilise qu'une voie d'un warp. De plus, à chaque itération, chaque thread lit un élément de A et un élément de B, et ceux de B sont espacés de M éléments en mémoire (une colonne de B), ce qui coûte cher en accès mémoire. Il aurait pu être pertinent de mesurer cet effet séparément mais je ne l'ai pas fait par contrainte de temps.

Par rapport au TP OpenMP : j'y avais mesuré à 1 thread environ 3 s à N = 1000 et 69 s à N = 2000 en double. Le GPU avec un thread par bloc fait 4.5 fois mieux à N = 1000 (0.666 s) et 13 fois mieux à N = 2000 (5.31 s). L'écart augmente avec N, sans doute parce que le CPU perd en efficacité quand les matrices dépassent ses caches, alors que le débit du GPU reste constant.

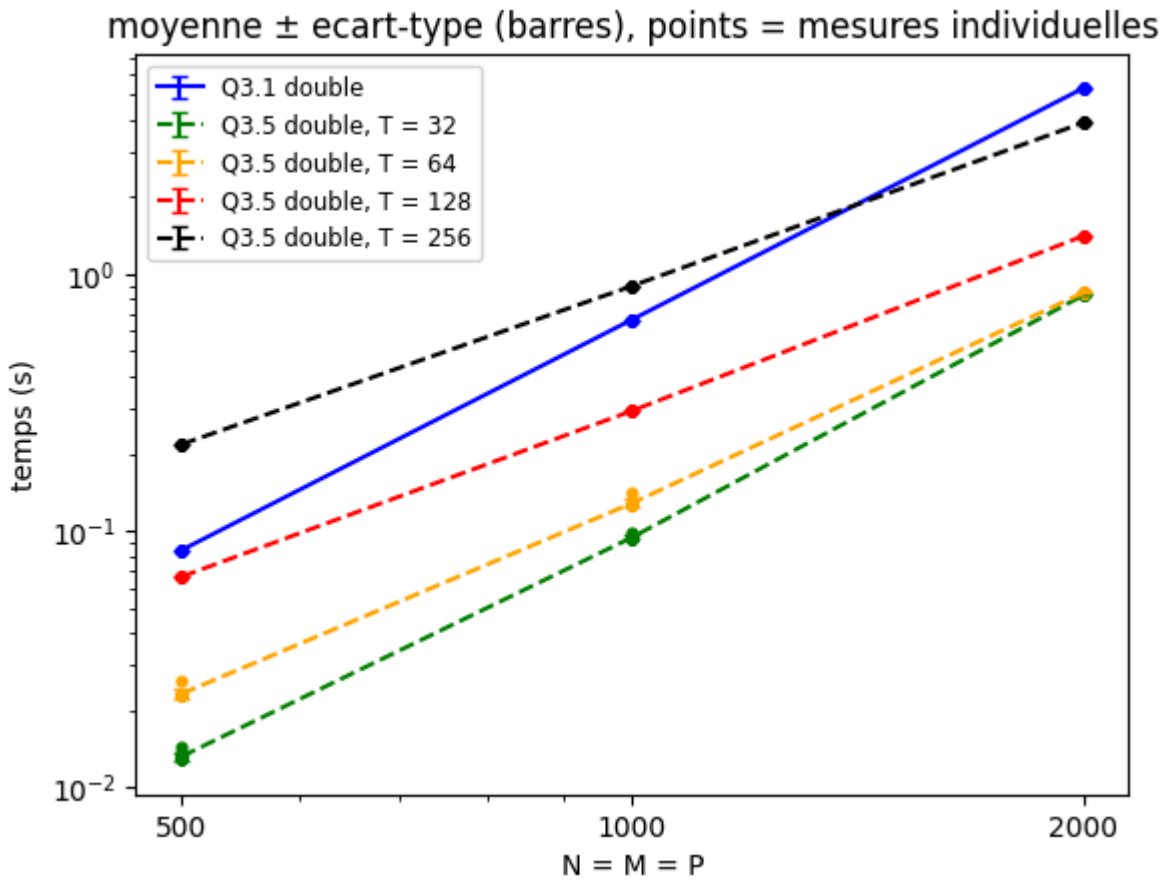
Question 3.5 - Plusieurs threads par bloc, mémoire partagée et atomiques

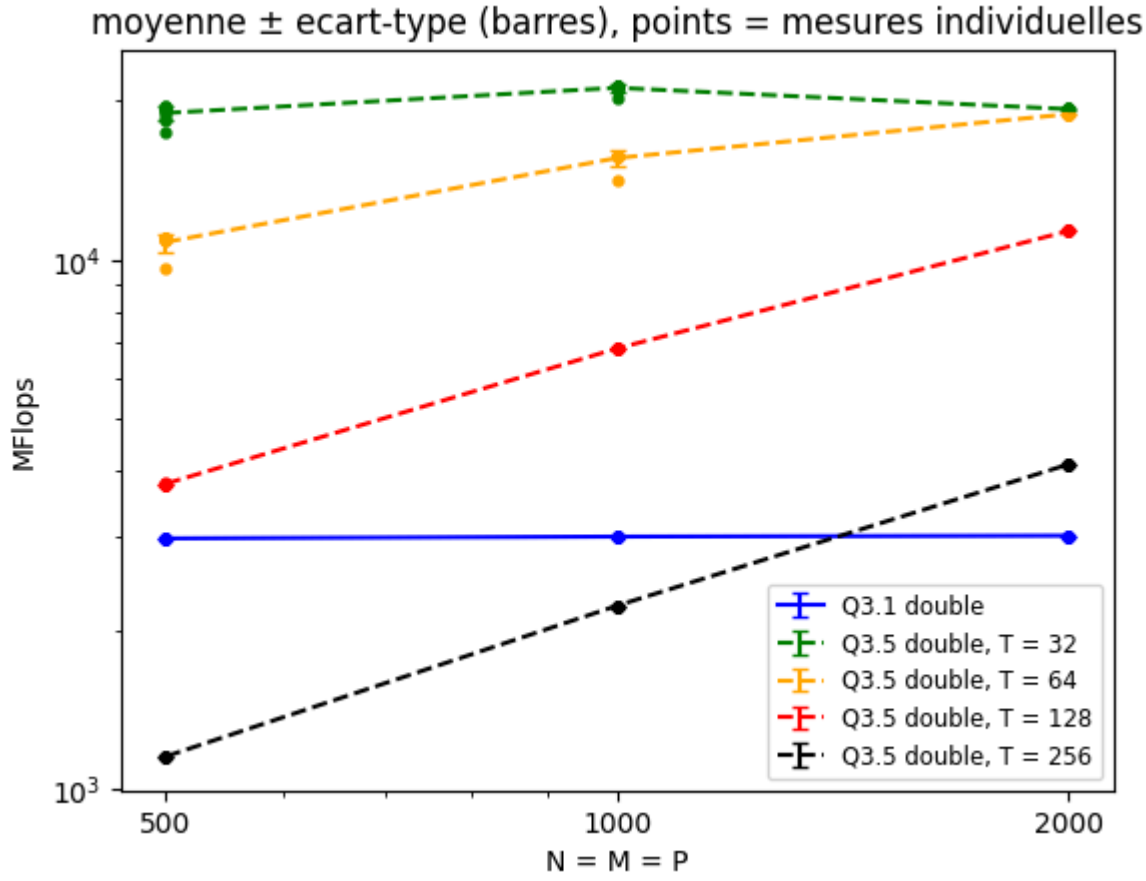
Le bloc (j, i) calcule toujours `C[i][j]`, mais avec `T` threads (argument `-T`) : le thread `t` traite les valeurs de k égales à `t`, `t + T`, `t + 2T`, etc. Chaque thread ajoute sa somme partielle à une variable en mémoire partagée avec `atomicAdd`, puis le thread 0 écrit le total dans `C`. Comme d'habitude, les deux `__syncthreads()` garantissent que le total est initialisé avant les ajouts et complet avant sa lecture.

Question 3.6 - Évaluation de performance (plusieurs threads par bloc)

Mêmes tailles qu'en 3.2, avec T = 32, 64, 128 et 256 threads par bloc et 10 répétitions. Les résultats s'ajoutent à la feuille `partie3_matrices` du classeur (elle n'est pas effacée ici).

Question 3.7 - Figures de performance (plusieurs threads par bloc)





Question 3.8 - Discussion (plusieurs threads par bloc)

Q3.5 (double), temps en secondes, 10 exécutions par configuration

threads par bloc	N = 500	N = 1000	N = 2000
32	0.01319 ± 0.00046	0.0945 ± 0.0017	0.82893 ± 0.00052
64	0.0231 ± 0.0010	0.1283 ± 0.0044	0.84735 ± 0.00017
128	0.06616 ± 0.00007	0.29372 ± 0.00010	1.40943 ± 0.00011
256	0.21686 ± 0.00007	0.90014 ± 0.00024	3.89236 ± 0.00029

Passer à plusieurs threads par bloc apporte un gain, mais seulement pour les petites valeurs de T. Avec T = 32, la Q3.5 est 6.4 fois plus rapide que la Q3.1 à N = 2000 (0.829 s contre 5.31 s) et 7.0 fois plus rapide à N = 1000 (0.0945 s contre 0.666 s), soit environ 19 GFlops à N = 2000. Ensuite le temps augmente avec T. À N = 2000, il passe de 0.829 s à T = 32 à 3.89 s à T = 256, et c'est encore plus net pour les petites matrices (de 0.0132 s à 0.217 s à N = 500, soit 16 fois plus). À T = 256 et N = 2000, la Q3.5 reste quand même 1.4 fois plus rapide que la Q3.1.

Comme en partie 2, je pense que la cause est la contention sur les additions atomiques. Chaque bloc fait T additions atomiques en mémoire partagée sur la même variable, soit N x M x T = 1.0 x 10^9 additions à N = 2000 et T = 256, et l'addition atomique d'un double en mémoire partagée est probablement émulée par une boucle de comparaison et d'échange. À T = 256 et N = 500, chaque thread ne fait qu'environ 2 itérations de la boucle sur k (500 / 256), donc les additions atomiques dominent le calcul. À T = 32 et T = 64, les temps sont presque identiques à N = 2000 (0.829 s et 0.847 s), et T = 32 est meilleur aux petites tailles. On aurait pu vérifier en mesurant séparément la part des additions atomiques.

Les écarts-types sont très faibles à N = 2000 (moins de 0.1 % du temps) et atteignent 2 à 4 % à N = 500 et N = 1000 pour T = 32 et T = 64.

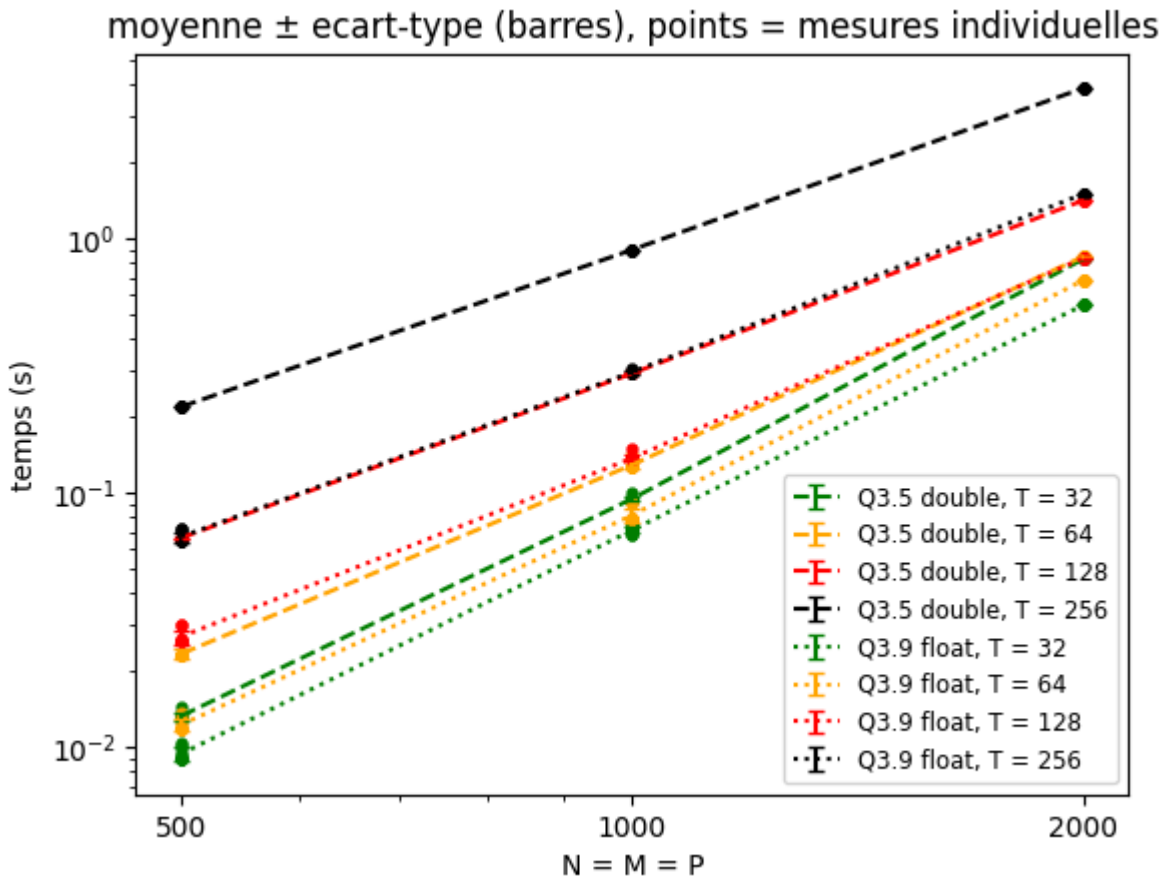
Question 3.9 - Passage de `double` à `float`

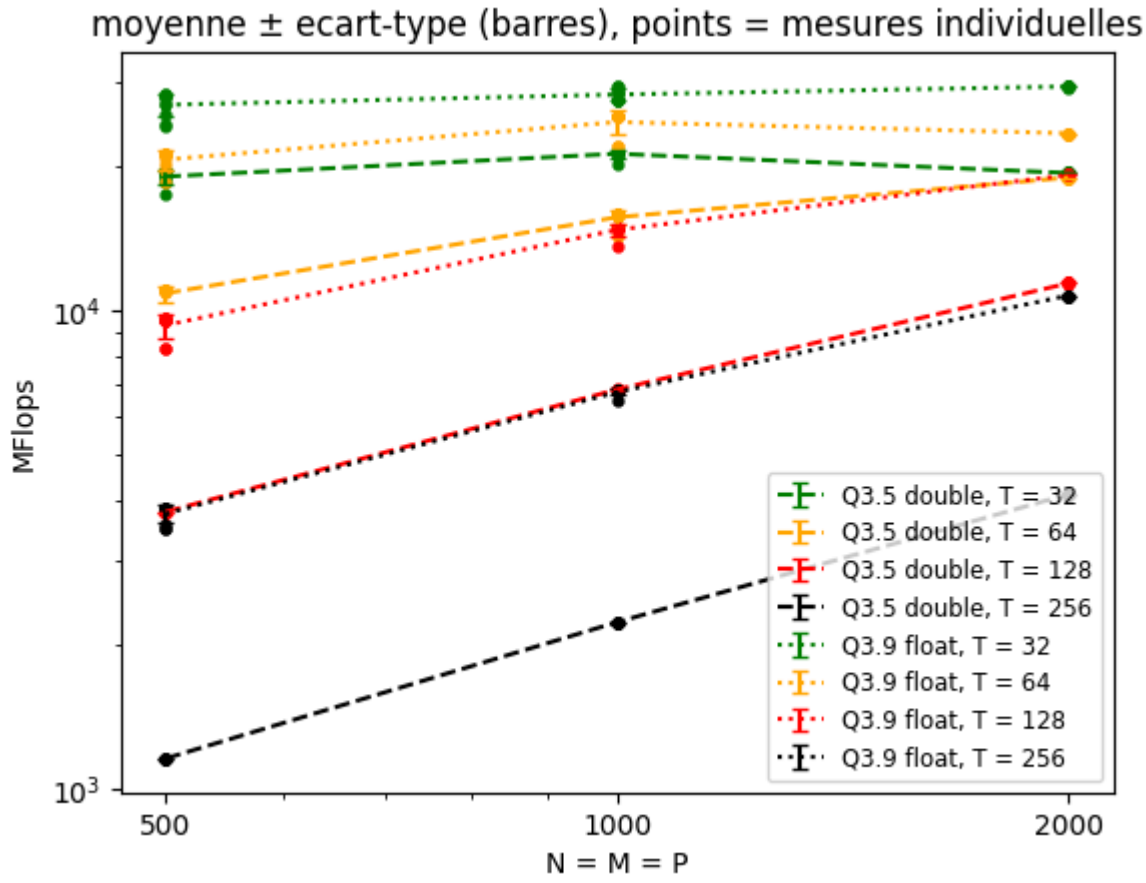
Les matrices, le kernel et la variable partagée passent en `float`. L'`atomicAdd` sur un `float` existe en matériel, contrairement à celui sur un `double` en mémoire partagée. Comme dans la version OpenMP en `float`, la vérification se fait par erreur relative (seuil 0.001) et non par la somme des carrés des écarts.

Question 3.10 - Évaluation de performance (float)

Mêmes tailles et mêmes nombres de threads par bloc qu'en 3.6, avec 10 répétitions. Les résultats s'ajoutent à la feuille `partie3_matrices` du classeur (elle n'est pas effacée ici).

Question 3.11 - Figures de performance (float)





Question 3.12 - Discussion (float)

Q3.9 (float), temps en secondes, 10 exécutions par configuration

threads par bloc	N = 500	N = 1000	N = 2000
32	0.00936 ± 0.00056	0.0711 ± 0.0019	0.5467 ± 0.0014
64	0.01217 ± 0.00060	0.0814 ± 0.0052	0.68465 ± 0.00016
128	0.0270 ± 0.0017	0.1363 ± 0.0041	0.83680 ± 0.00018
256	0.0669 ± 0.0030	0.2974 ± 0.0042	1.49446 ± 0.00013

Rapport du temps double (Q3.5) / float (Q3.9)

threads par bloc	N = 500	N = 1000	N = 2000
32	1.41	1.33	1.52
64	1.90	1.58	1.24
128	2.45	2.15	1.68
256	3.24	3.03	2.60

Le passage en float accélère toutes les configurations. Avec T = 32, le temps passe de 0.829 s en double à 0.547 s à N = 2000, soit 1.52 fois moins, et le rapport double sur float va de 1.24 à 3.24 selon T et N. Le gain est le plus fort à grand T : 2.60 fois à T = 256 et N = 2000, et 3.24 fois à N = 500. 2 hypothèses :

- Les matrices en float occupent deux fois moins de mémoire, donc il y a deux fois moins de données à lire.
- L'addition atomique d'un float existe en matériel, contrairement à celle d'un double en mémoire partagée, ce qui réduit le coût de la contention quand T est grand.

Le gain plus fort à T = 256 qu'à T = 32 va dans le sens de la seconde, mais je n'ai pas séparé les deux.

Le meilleur débit en float est de 29.3 GFlops (T = 32, N = 2000), très en dessous des environ 3 TFlops de la carte en simple précision. Le kernel lit B colonne par colonne sans réutilisation en mémoire partagée, donc il est probablement limité par les accès mémoire et par les additions atomiques. Comme en double, T = 32 est la meilleure valeur et le temps augmente avec T (de 0.547 s à 1.494 s à N = 2000), mais moins vite qu'en double (facteur 2.7 contre 4.7).

Question 3.13 - Passage de `double` à `half`

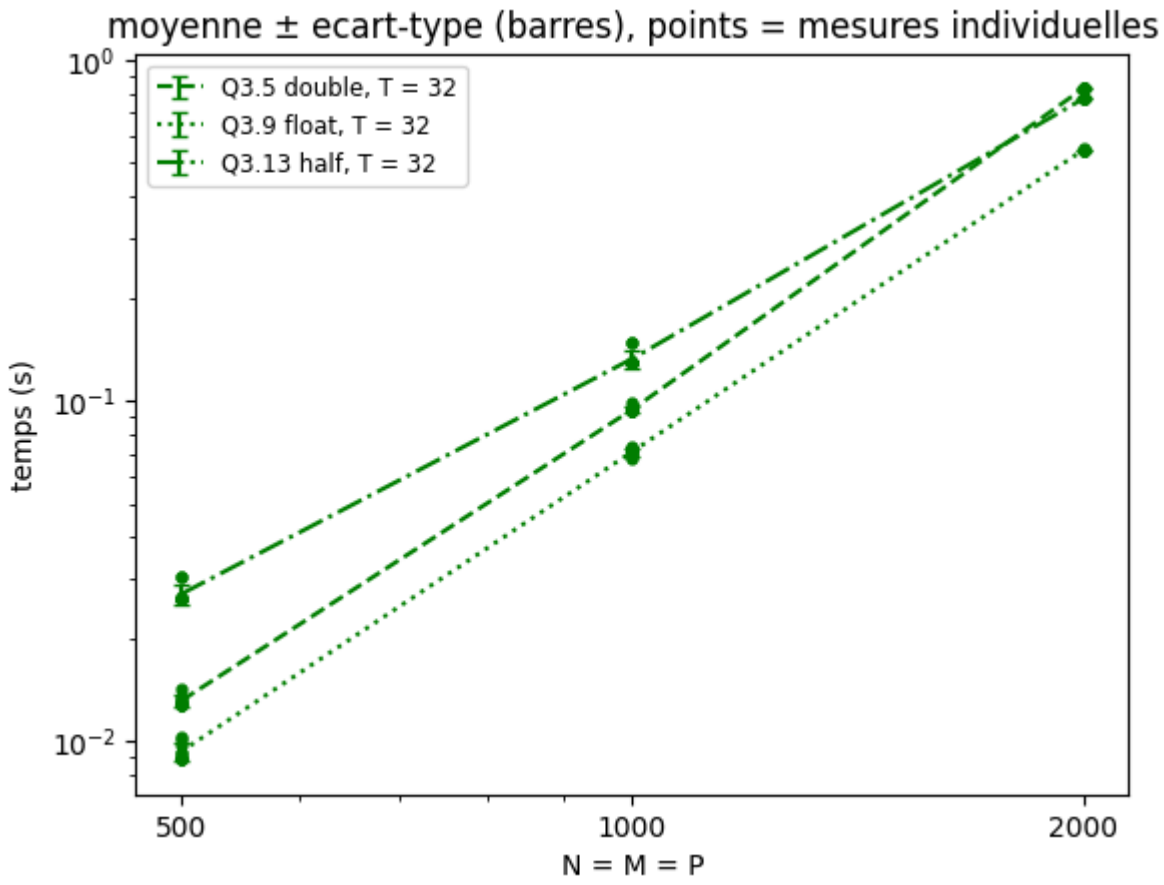
L'énoncé propose d'inclure `half.hpp` et d'utiliser `half_float::half`. Cette bibliothèque ne fonctionne que sur le processeur : ses opérations ne sont pas utilisables dans un kernel CUDA. J'utilise donc le type `__half` de CUDA (`cuda_fp16.h`), qui joue le même rôle sur le GPU. Les matrices `A`, `B` et `C`, la variable partagée et la somme partielle de chaque thread sont en `__half`, comme dans la version OpenMP en `half`, avec les opérations `__hmul` et `__hadd`. La vérification se fait par erreur relative avec un seuil de 0.05, comme au TP OpenMP.

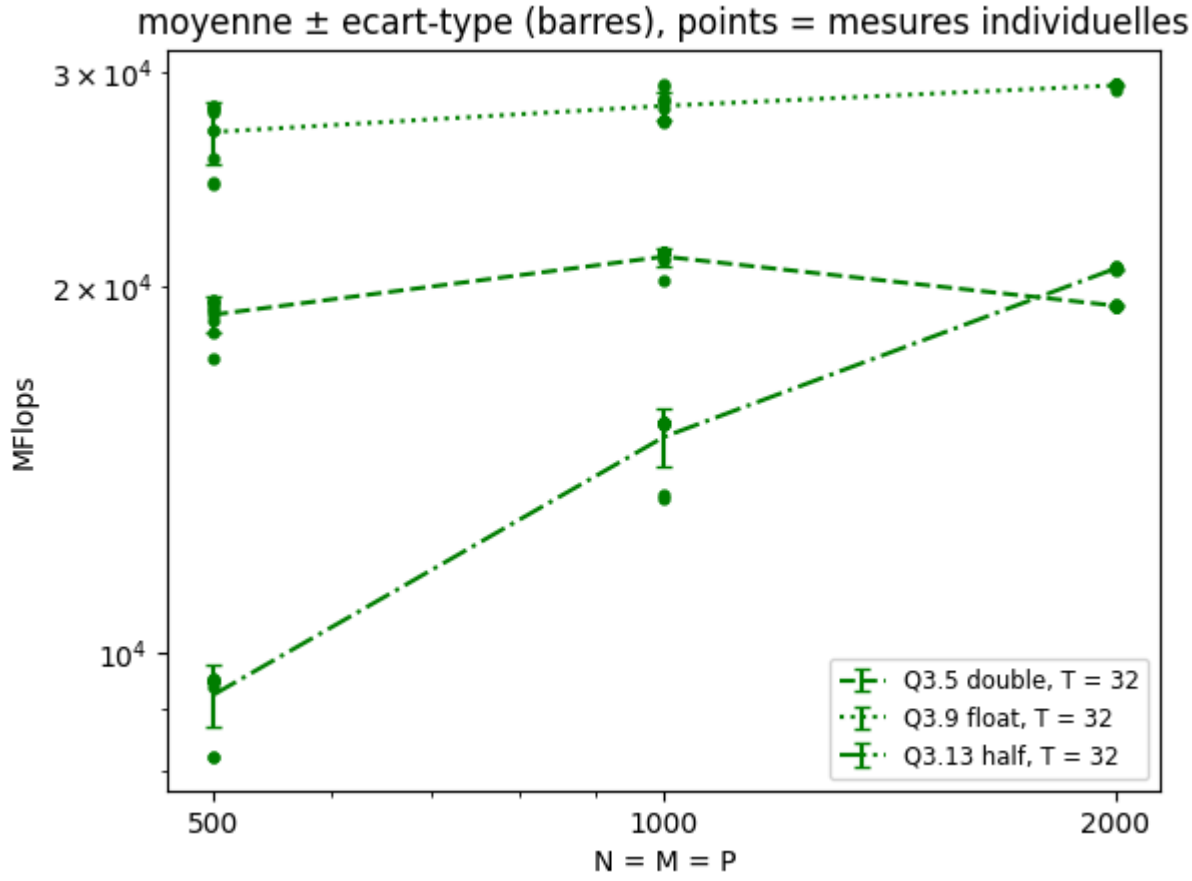
Un `half` ne dépasse pas 65504. Chaque élément du produit vaut environ 17 x P, donc le résultat déborde dès que P dépasse environ 3850 : pour N = M = P = 4000, le programme affiche « Errors in multiplication ». Les tailles de 500 à 2000 restent en dessous de cette limite.

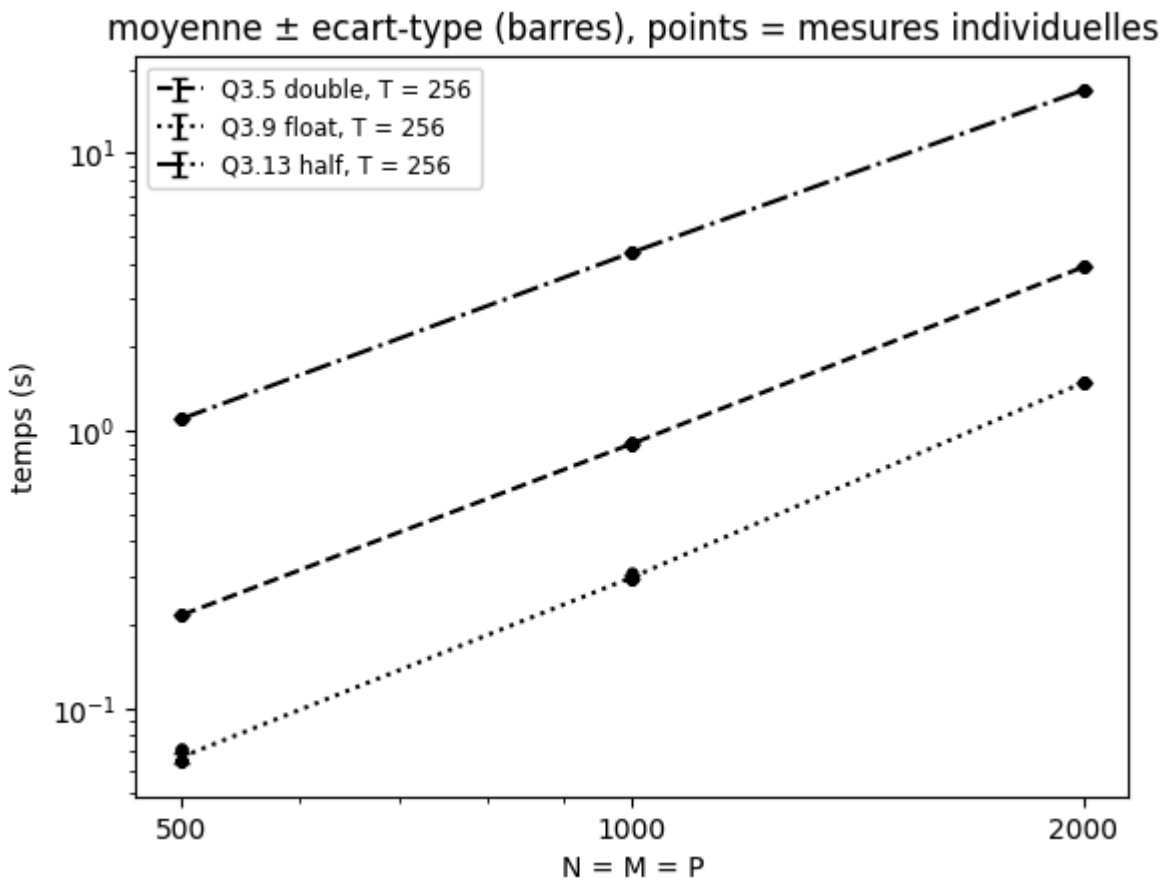
Question 3.14 - Évaluation de performance (half)

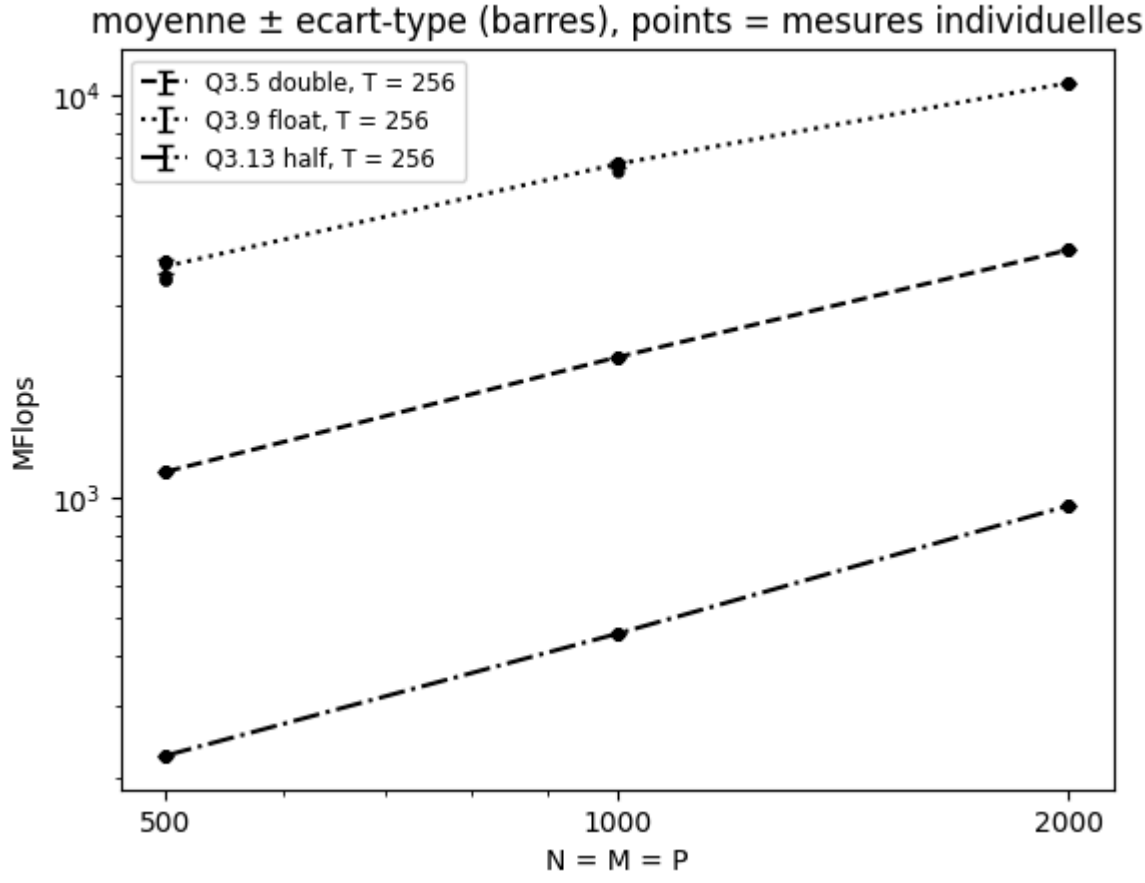
Mêmes tailles et mêmes nombres de threads par bloc qu'en 3.6 et 3.10, avec 10 répétitions. Les résultats s'ajoutent à la feuille `partie3_matrices` du classeur (elle n'est pas effacée ici).

Question 3.15 - Figures de performance (half)






moyenne ± ecart-type (barres), points = mesures individuelles
Q3.5 double, T = 256
Q3.9 float, T = 256
Q3.13 half, T = 256
temps (s)
10^1
10^0
10^{-1}
500
1000
2000
N = M = P


moyenne ± ecart-type (barres), points = mesures individuelles
Q3.5 double, T = 256
Q3.9 float, T = 256
Q3.13 half, T = 256
MFlops
10^4
10^3
500
1000
2000
N = M = P

Question 3.16 - Discussion (half)

Q3.13 (half), temps en secondes, 10 exécutions par configuration

threads par bloc	N = 500	N = 1000	N = 2000
32	0.0271 ± 0.0017	0.1332 ± 0.0081	0.7728 ± 0.0012
64	0.0749 ± 0.0047	0.3045 ± 0.0012	1.34947 ± 0.00041
128	0.27774 ± 0.00039	1.11978 ± 0.00026	4.55900 ± 0.00035
256	1.10327 ± 0.00021	4.39104 ± 0.00023	16.85678 ± 0.00095

Rapport du temps half (Q3.13) / float (Q3.9)

threads par bloc	N = 500	N = 1000	N = 2000
32	2.90	1.87	1.41
64	6.16	3.74	1.97
128	10.27	8.21	5.45
256	16.49	14.77	11.28

Le half n'apporte aucun gain de vitesse ici, alors qu'on pourrait l'attendre d'un type deux fois plus petit que le float. Avec T = 32, il est 1.41 fois plus lent que le float à N = 2000 (0.773 s contre 0.547 s) et 2.90 fois plus lent à N = 500, et l'écart explose avec T : à T = 256, il est 11.3 fois plus lent à N = 2000 (16.86 s contre 1.49 s) et 16.5 fois plus lent à N = 500. Par rapport au double, il est à peine plus rapide à T = 32 (0.773 s contre 0.829 s à N = 2000) et beaucoup plus lent dès T = 128 (4.56 s contre 1.41 s).

Je vois deux raisons possibles, que je n'ai pas vérifiées. D'abord, d'après mes questions à une LLM, les paires de half traitées ensemble profitent d'un débit doublé offert par la carte (`half2`). Mais cet avantage ne sera donc pas présent pour les opérations sur un half isolé (`__hmul` et `__hadd`). Ensuite, l'addition atomique d'un half en mémoire partagée doit sûrement encore être émulée par une boucle de comparaison et d'échange, plus coûteuse encore que celle du double, ce qui expliquerait l'effondrement pour un grand T. L'écart avec le float est plus fort sur les petites matrices (16.5 fois à N = 500 contre 11.3 fois à N = 2000). Ça va dans le sens d'un coût fixe par bloc (les additions atomiques) qui pèse plus quand le calcul est court.

Le half est aussi limité en précision et en amplitude. Un élément du produit vaut environ 17 x P et un half ne dépasse pas 65504, donc le résultat déborde pour P supérieur à environ 3850 (j'ai observé « Errors in multiplication » à N = 4000). Aux tailles mesurées, qui vont jusqu'à 2000 (éléments d'environ 34 000), la vérification par erreur relative avec un seuil de 5 % a réussi dans mes essais (N = 500 avec T = 1, 32, 128 et 256, puis N = 1000 et 2000 avec T = 32).

En conclusion, pour cette application, je choisirais le float : plus rapide que le double et sans les limites du half.

4. Chaining Matrix Multiplication

Question 4.1 - Implémentation parallèle du workflow

Le graphe est le même que celui du TP OpenMP : 4 produits indépendants (AB, CD, EF, GH), puis 2 produits (ABCD et EFGH), puis le produit final, soit 7 produits sur 3 niveaux. Chaque produit réutilise le kernel de la question 3.5 (plusieurs threads par bloc, mémoire partagée et `atomicAdd`), lancé dans un stream CUDA. Les produits d'un même niveau sont répartis sur les streams disponibles (`flux[k % nstreams]`), et ils peuvent donc s'exécuter en même temps. Pour qu'un produit n'attende que les deux résultats dont il dépend, chaque produit enregistre un événement (`cudaEventRecord`) à sa fin, et le stream du produit suivant l'attend avec `cudaStreamWaitEvent`, sans bloquer le CPU.

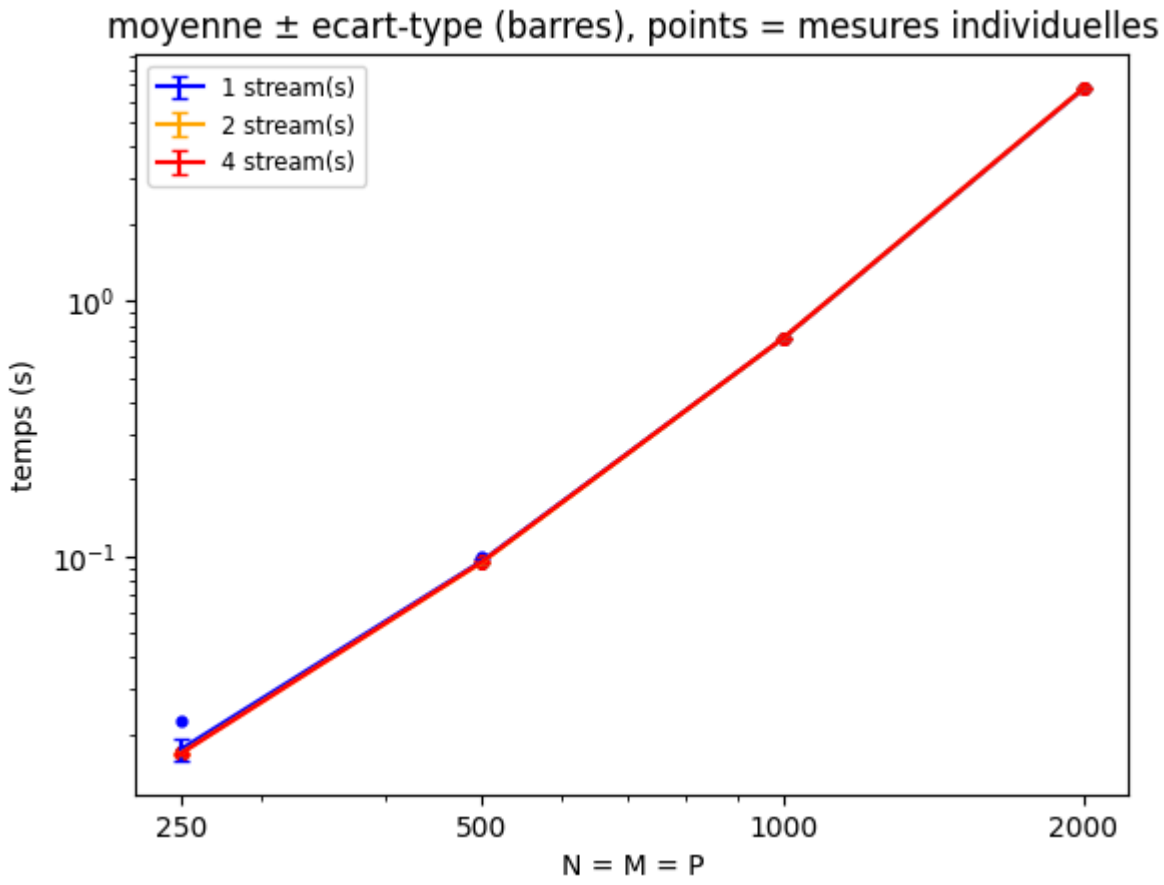
Le nombre de streams est un argument (`-S`). Avec `-S 1`, les 7 produits passent dans le même stream et s'exécutent l'un après l'autre : c'est la référence séquentielle, avec exactement le même code que la version à plusieurs streams. Les 8 matrices d'entrée valent `AVAL`, donc chaque case du résultat vaut N^7 x AVAL^8, et la vérification porte sur l'erreur relative maximale (seuil 1e-9), comme dans la version OpenMP. Le temps comprend le lancement des 7 produits et l'attente de la fin du dernier (`cudaDeviceSynchronize`), mais pas les copies vers et depuis le GPU. Les MFlops sont calculés avec 7 x 2 x N^3 opérations.

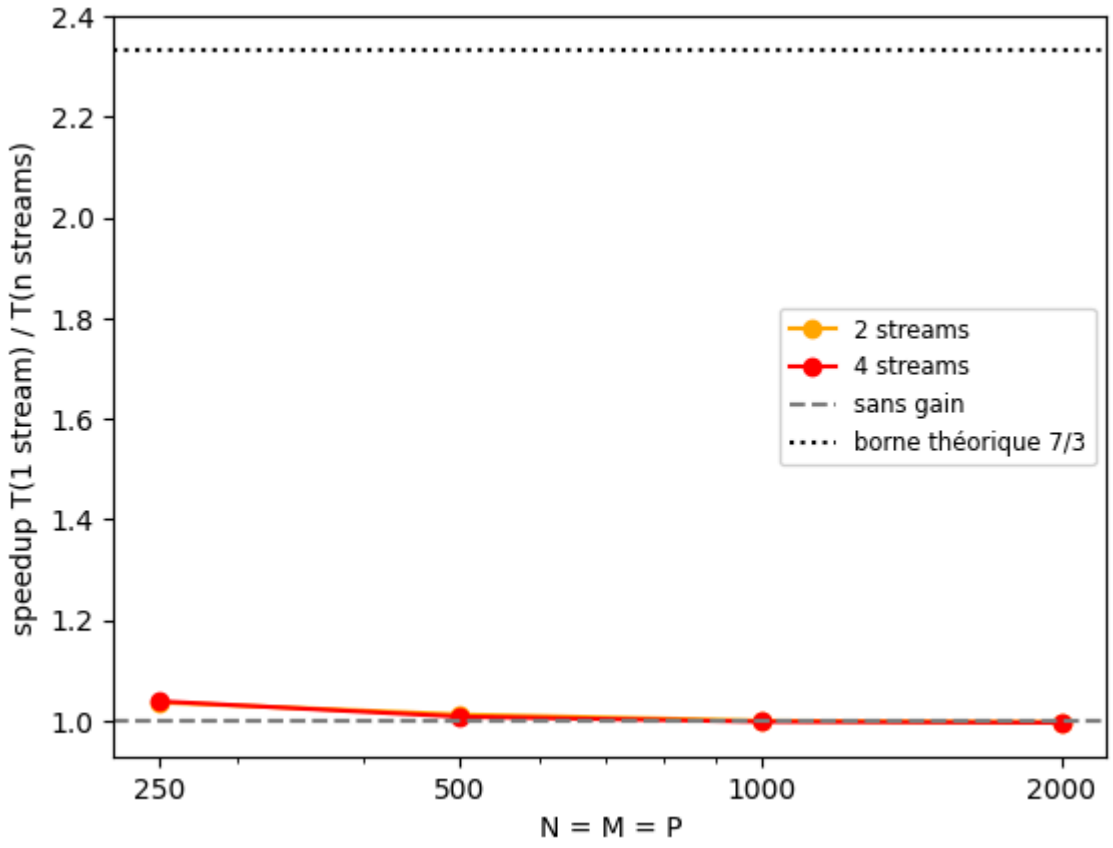
Question 4.2 - Évaluation de performance

L'énoncé demande N et M de 1000 à 18000. Les matrices sont carrées, donc je donne la même valeur à N, M et P. Le workflow garde 15 matrices en mémoire (8 entrées, 7 résultats) : avec 4 Go, N = 18000 est impossible (environ 39 Go en `double`) et N = 8000 aussi (environ 7.7 Go). N = 4000 tient (environ 1.9 Go), mais une exécution coûterait environ 76 s d'après la partie 3 (7 produits de 11 s), soit plus de 20 minutes par valeur du nombre de streams avec 10 répétitions. La grille est donc réduite à N = M = P parmi 250, 500, 1000 et 2000, avec 1, 2 et 4 streams et 10 répétitions. Le nombre de threads par bloc est fixé à 32, la meilleure valeur de la partie 3.

N = M = P	streams	répétitions
250, 500, 1000, 2000	1, 2, 4	10

Question 4.3 - Figures de performance





Question 4.4 - Discussion

Temps de la chaîne de 7 produits en secondes, T = 32, 10 exécutions par configuration

N = M = P	1 stream	2 streams	4 streams
250	0.0175 ± 0.0017	0.01692 ± 0.00014	0.01688 ± 0.00017
500	0.0957 ± 0.0015	0.09452 ± 0.00049	0.09494 ± 0.00072
1000	0.7105 ± 0.0015	0.7107 ± 0.0013	0.7121 ± 0.0016
2000	6.770 ± 0.011	6.7906 ± 0.0054	6.7992 ± 0.0075

Speedup T(1 stream) / T(n streams) et borne théorique 7/3 = 2.33

N = M = P	2 streams	4 streams
250	1.036	1.038
500	1.012	1.008
1000	1.000	0.998
2000	0.997	0.996

Les streams n'apportent aucun gain mesurable. Le speedup avec 4 streams est de 1.038 à N = 250 (0.01753 s avec 1 stream contre 0.01688 s avec 4), puis de 1.008 à N = 500, de 0.998 à N = 1000 et de 0.996 à N = 2000, très loin de la borne théorique de 7/3 qu'on obtiendrait si les produits d'un même niveau s'exécutaient vraiment en parallèle. À N = 250, l'écart de 4 % est du même ordre que l'écart-type de la version à 1 stream (0.0017 s, soit 10 %), donc il n'est pas significatif. À N = 2000, la version à 4 streams est même 0.4 % plus lente (6.799 s contre 6.770 s), ce qui est à peine plus que les écarts-types.

Mon interprétation est que chaque produit occupe déjà toute la carte. Un produit lance N x M blocs de 32 threads, soit 62 500 blocs à N = 250 et 4 millions à N = 2000, alors que la carte ne peut en exécuter qu'environ 224 à la fois (14 multiprocesseurs, avec au plus 16 blocs résidents par multiprocesseur d'après les spécifications de cette architecture). Un seul kernel suffit donc à saturer le GPU, et un second kernel lancé dans un autre stream ne peut démarrer que quand des blocs du premier se terminent, c'est-à-dire en fin de kernel. On pourrait éventuellement vérifier ce point avec un outil de profilage, par exemple en observant la chronologie des kernels.

Dans le TP OpenMP, le speedup de l'enchaînement plafonnait autour de 2, avec la même borne de 7/3, parce que chaque produit était séquentiel sur un seul cœur : le parallélisme entre les produits d'un niveau était alors la seule source de parallélisme disponible. Ici, chaque produit est déjà très parallèle, et celui entre produits n'ajoute rien. Pour que les streams aident, il faudrait des produits qui n'occupent qu'une partie de la carte, par exemple avec peu de blocs, mais je ne l'ai pas testé.

Le temps de la chaîne est proche de 7 fois celui d'un produit isolé de la partie 3 (T = 32) : 0.0957 s contre 7 x 0.0132 = 0.0923 s à N = 500, 0.710 s contre 0.661 s à N = 1000 et 6.77 s contre 5.80 s à N = 2000. La chaîne est donc de 4 % à 17 % plus lente que sept produits isolés, et l'écart augmente avec N. Je n'ai pas d'explication vérifiée. Peut-être que la mémoire occupée (15 matrices, environ 480 Mo à N = 2000) rend la lecture des matrices moins efficace que dans la partie 3, mais je ne l'ai pas mesuré. Le débit de la chaîne reste de l'ordre de 12 à 20 GFlops selon la taille, comparable à celui d'un produit seul.